



Lecture 9

Module 5: Networking and Content Delivery

Module 6: Compute



Module 5: Networking and Content Delivery

AWS Academy Cloud Foundations



Module overview

Topics

- Networking basics
- Amazon VPC
- VPC networking
- VPC security
- Amazon Route 53
- Amazon CloudFront

Activities

- Label a network diagram

Demo

- VPC demonstration

Lab

- Build your VPC and launch a web server





Module objectives

After completing this module, you should be able to:

- Recognize the basics of networking
- Describe virtual networking in the cloud with Amazon VPC
- Label a network diagram
- Design a basic VPC architecture
- Indicate the steps to build a VPC
- Identify security groups
- Create your own VPC and add additional components to it to produce a customized network
- Identify the fundamentals of Amazon Route 53

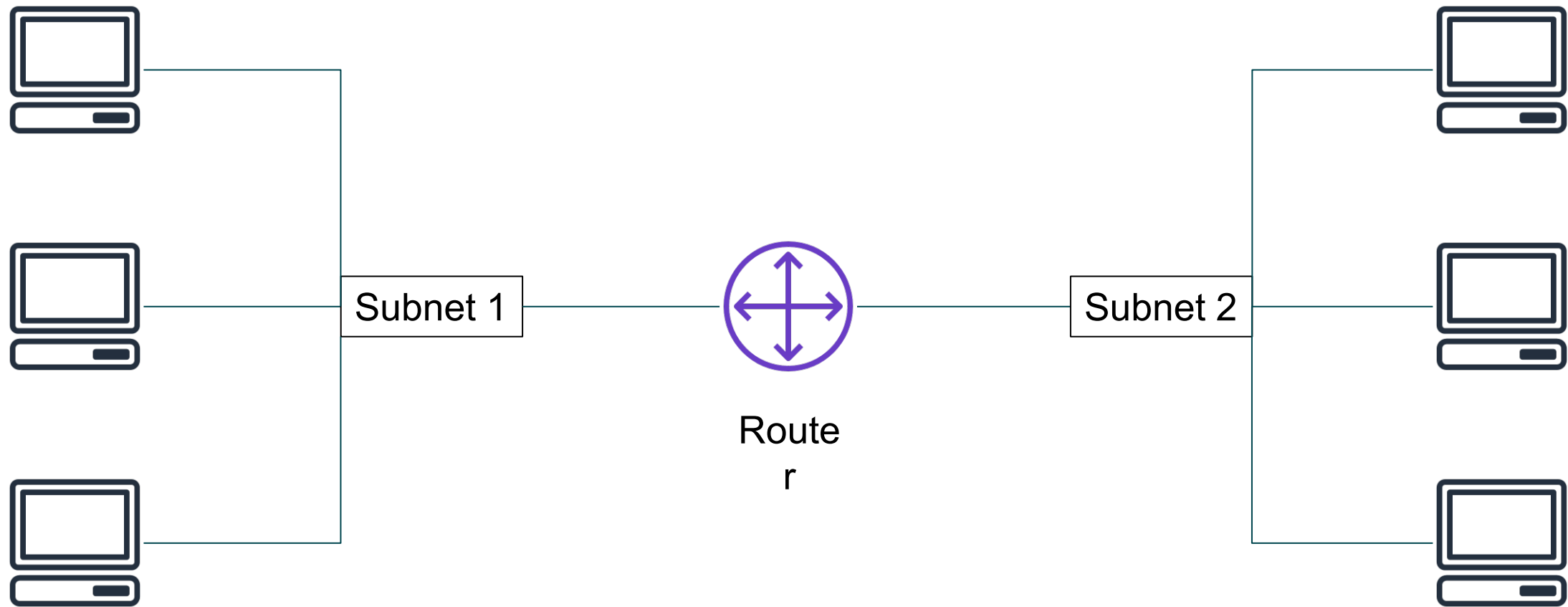
 Recognize the benefits of Amazon CloudFront



Section 1: Networking basics

Module 5: Networking and Content Delivery

Networks





IP addresses

192

.

0

.

2

.

0



11000000

00000000

00000010

00000000



IPv4 and IPv6 addresses

IPv4 (32-bit) address:

192.0.2.0

IPv6 (128-bit) address:

2600:1f18:22ba:8c00:ba86:a05e:a5ba:00FF

Classless Inter-Domain Routing (CIDR)

Network identifier (routing prefix)

192 . 0 . 2

Host identifier

. 0 /

24

↓
11000000

Fixed

↓
00000000

Fixed

↓
00000010

Fixed

↓
00000000
to 11111111

Flexible

Tells you
how
many bits
are
fixed



Open Systems Interconnection (OSI) model

Layer	Number	Function	Protocol/Address
Application	7	Means for an application to access a computer network	HTTP(S), FTP, DHCP, LDAP
Presentation	6	- Ensures that the application layer can read the data - Encryption	ASCII, ICA
Session	5	Enables orderly exchange of data	NetBIOS, RPC
Transport	4	Provides protocols to support host-to-host communication	TCP, UDP
Network	3	Routing and packet forwarding (routers)	IP
Data link	2	Transfer data in the same LAN network (hubs and switches)	MAC
Physical	1	Transmission and reception of raw bitstreams over a physical medium	Signals (1s and 0s)



Section 2: Amazon VPC

Module 5: Networking and Content Delivery





Amazon VPC

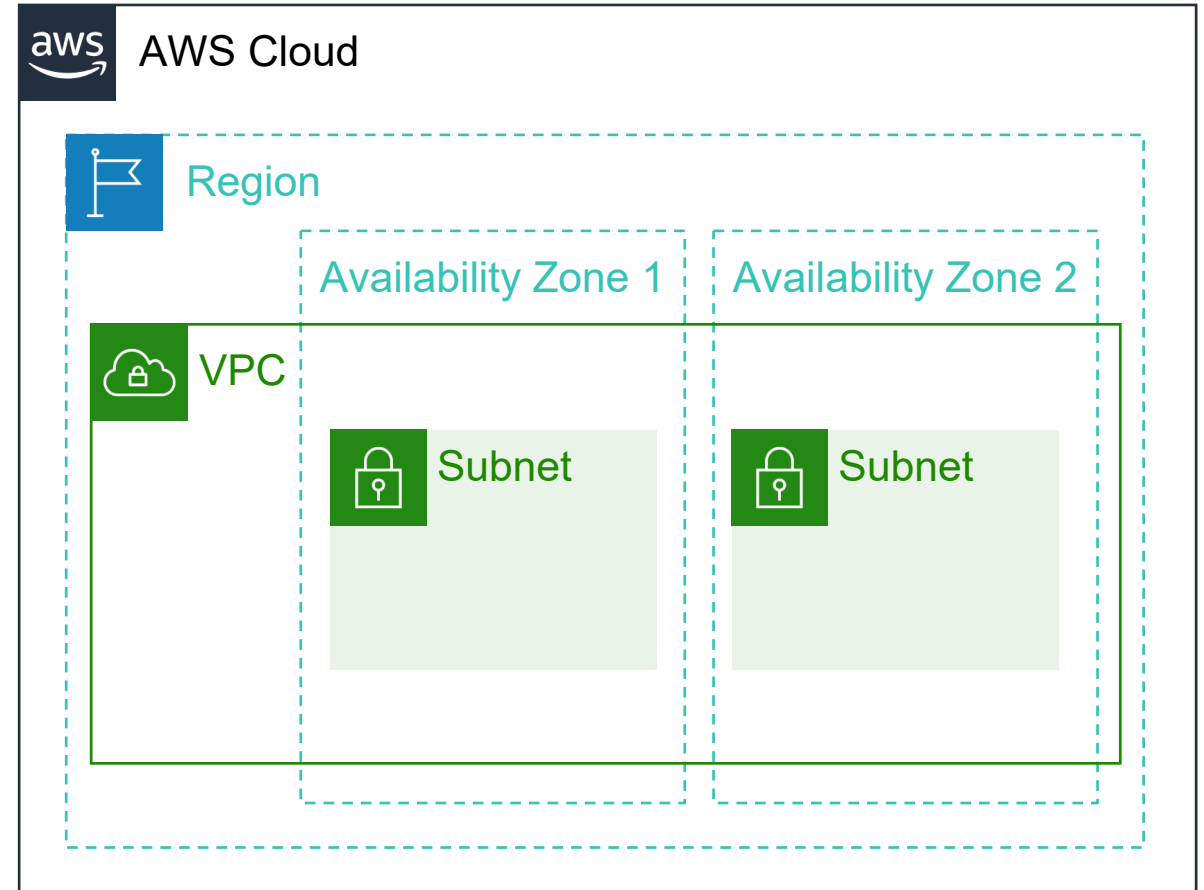


Amazon
VPC

- Enables you to provision a **logically isolated** section of the AWS Cloud where you can launch AWS resources in a virtual network that you define
- Gives you **control over your virtual networking resources**, including:
 - Selection of IP address range
 - Creation of subnets
 - Configuration of route tables and network gateways
- Enables you to **customize the network configuration** for your VPC
- Enables you to use **multiple layers of security**

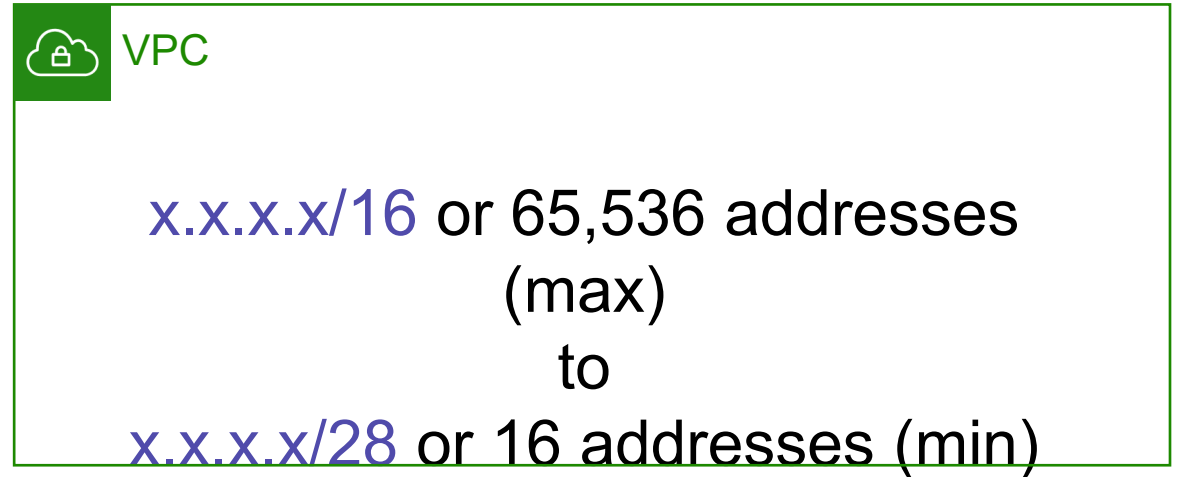
VPCs and subnets

- VPCs:
 - Logically isolated from other VPCs
 - Dedicated to your AWS account
 - Belong to a single AWS Region and can span multiple Availability Zones
- Subnets:
 - Range of IP addresses that divide a VPC
 - Belong to a single Availability Zone
 - Classified as public or private



IP addressing

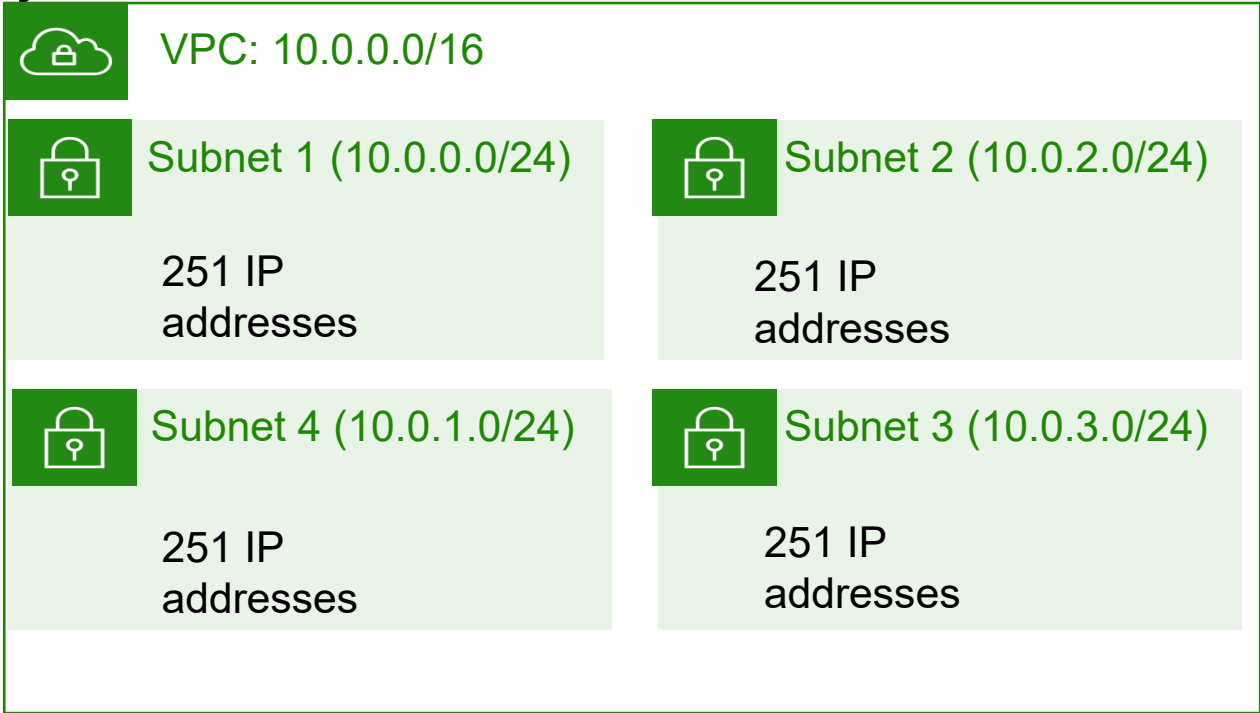
- When you create a VPC, you assign it to an IPv4 **CIDR block** (range of **private** IPv4 addresses).
- You **cannot change the address range** after you create the VPC.
- The **largest** IPv4 CIDR block size is **/16**.
- The **smallest** IPv4 CIDR block size is **/28**.
- IPv6 is also supported (with a different block size limit).
- CIDR blocks of subnets **cannot overlap**.



Reserved IP addresses

Example: A VPC with an IPv4 CIDR block of 10.0.0.0/16 has 65,536 total IP addresses.

The VPC has four equal-sized subnets. Only 251 IP addresses are available for use by each subnet.



IP Addresses for CIDR block 10.0.0.0/24	Reserved for
10.0.0.0	Network address
10.0.0.1	Internal communication
10.0.0.2	Domain Name System (DNS) resolution
10.0.0.3	Future use
10.0.0.255	Network broadcast address



Public IP address types

Public IPv4 address

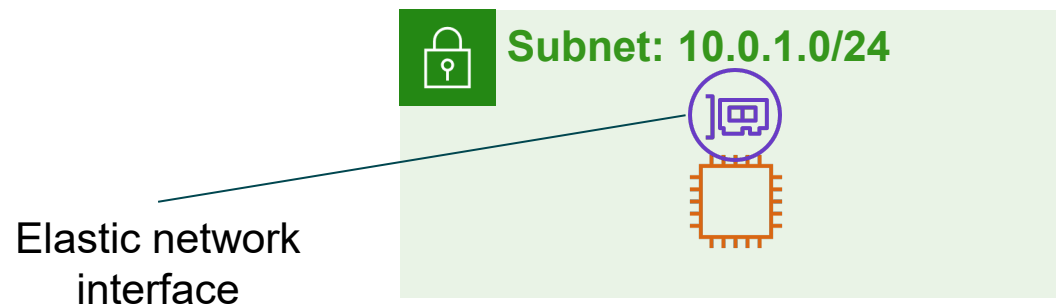
- Manually assigned through an Elastic IP address
- Automatically assigned through the auto-assign public IP address settings at the subnet level

Elastic IP address

- Associated with an AWS account
- Can be allocated and remapped anytime
- Additional costs might apply

Elastic network interface

- An elastic network interface is a **virtual network interface** that you can:
 - Attach to an instance.
 - Detach from the instance, and attach to another instance to redirect network traffic.
- Its **attributes follow** when it is reattached to a new instance.
- Each instance in your VPC has a **default network interface** that is assigned a private IPv4 address from the IPv4 address range of your VPC.





Route tables and routes

- A **route table** contains a set of rules (or routes) that **you can configure** to direct network traffic from your subnet.
- Each **route** specifies a destination and a target.
- By default, every route table contains a **local route** for communication within the VPC.
- Each **subnet must be associated with a route table** (at most one).

Main (Default) Route Table

Destination	Target
10.0.0.0/16	local

VPC CIDR
block

Section 2 key takeaways



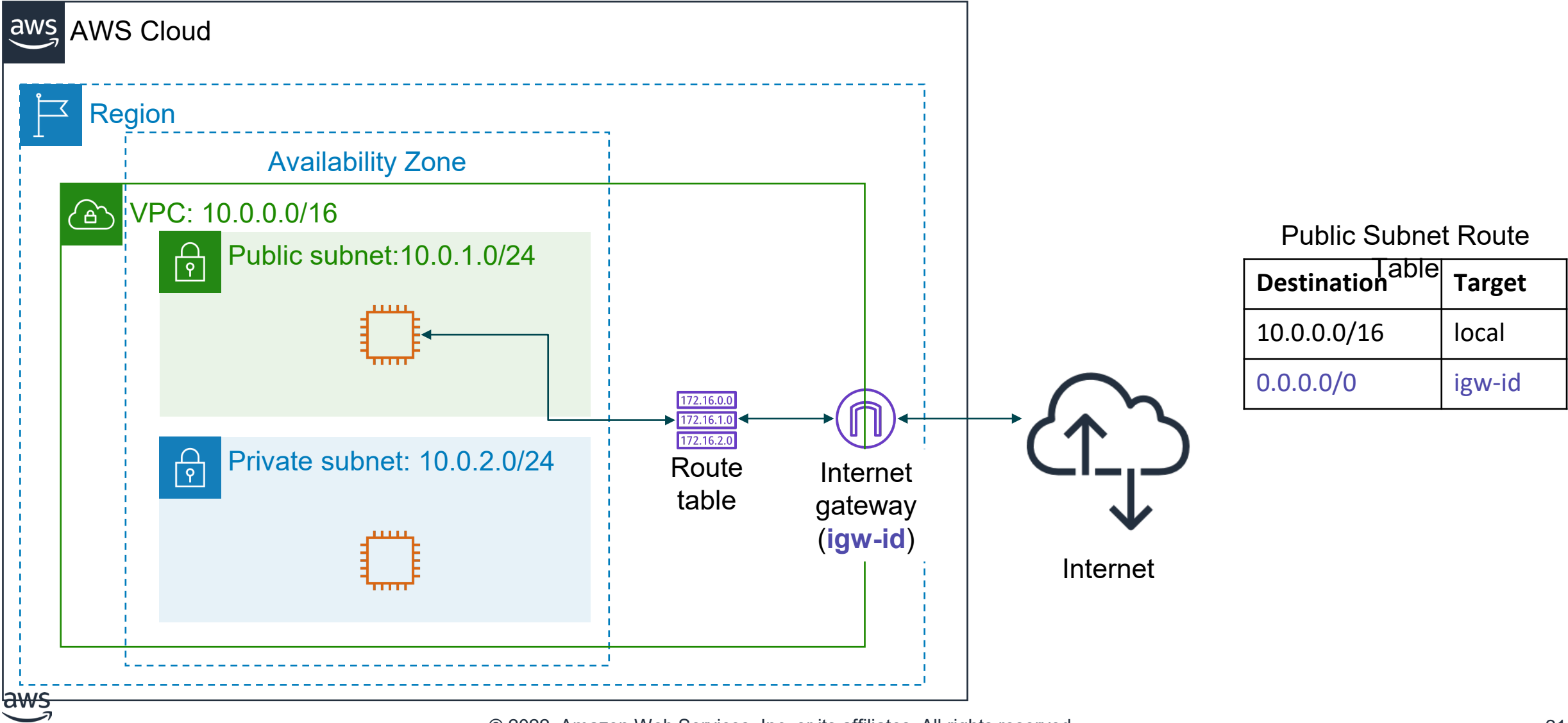
- A VPC is a logically isolated section of the AWS Cloud.
- A VPC belongs to one Region and requires a CIDR block.
- A VPC is subdivided into subnets.
- A subnet belongs to one Availability Zone and requires a CIDR block.
- Route tables control traffic for a subnet.
- Route tables have a built-in local route.
- You add additional routes to the table.
- The local route cannot be deleted.



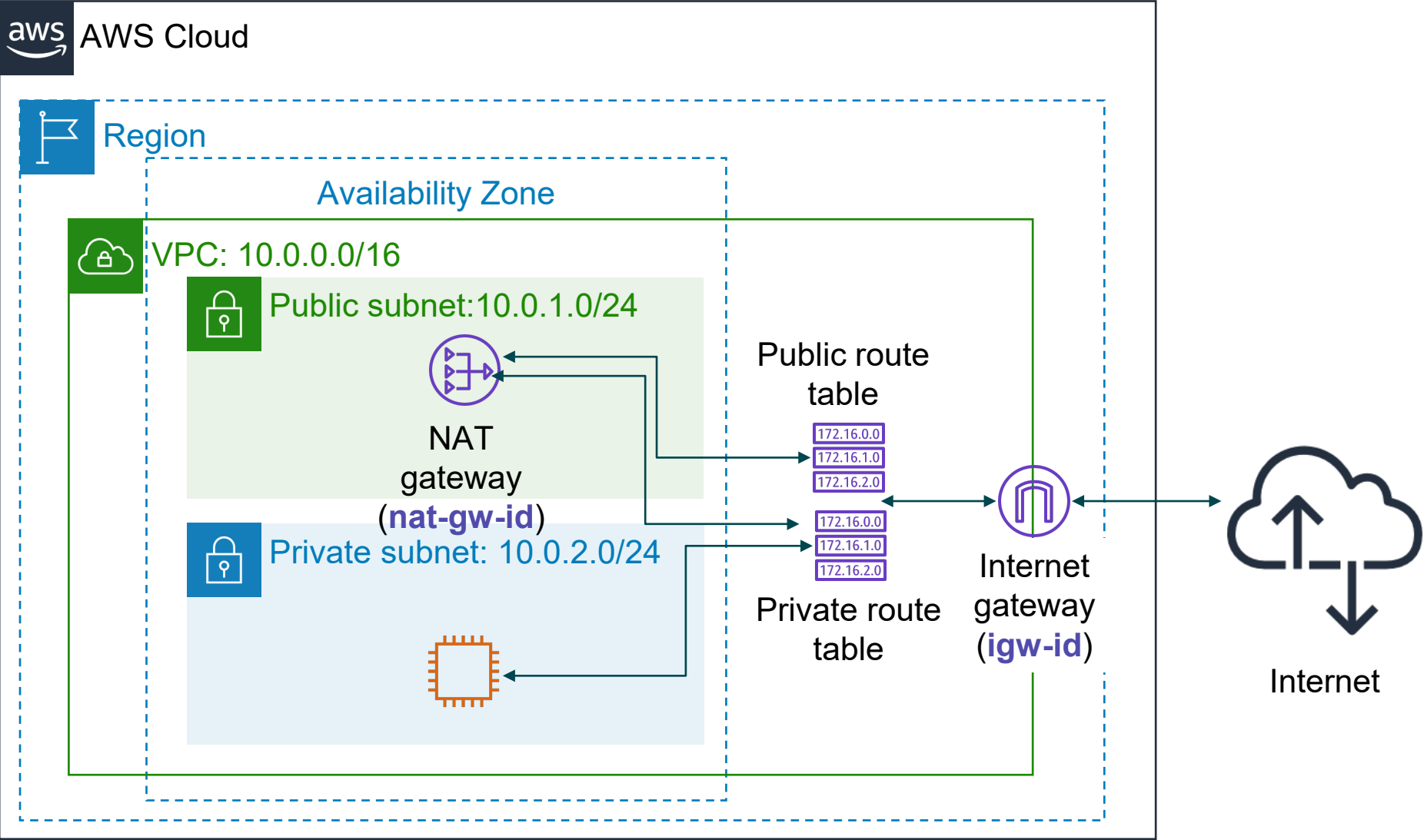
Section 3: VPC networking

Module 5: Networking and Content Delivery

Internet gateway



Network address translation (NAT) gateway



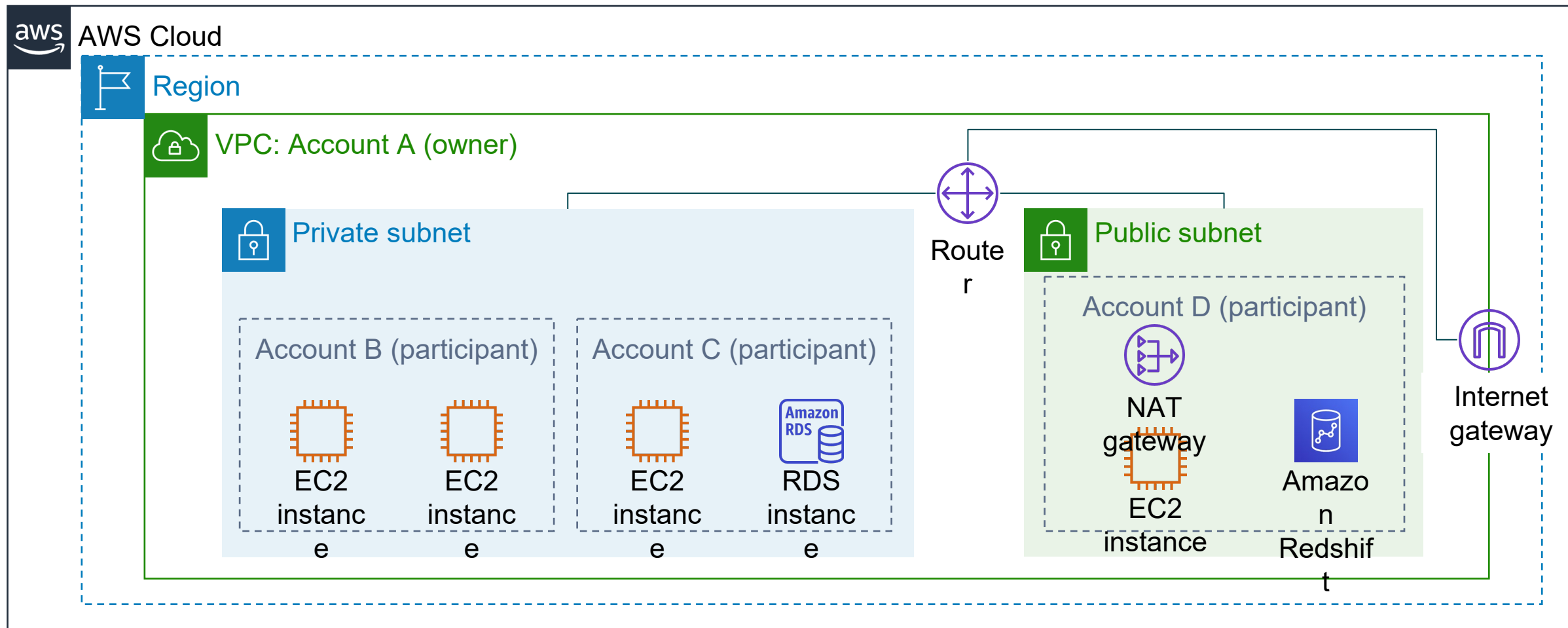
Public Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-id

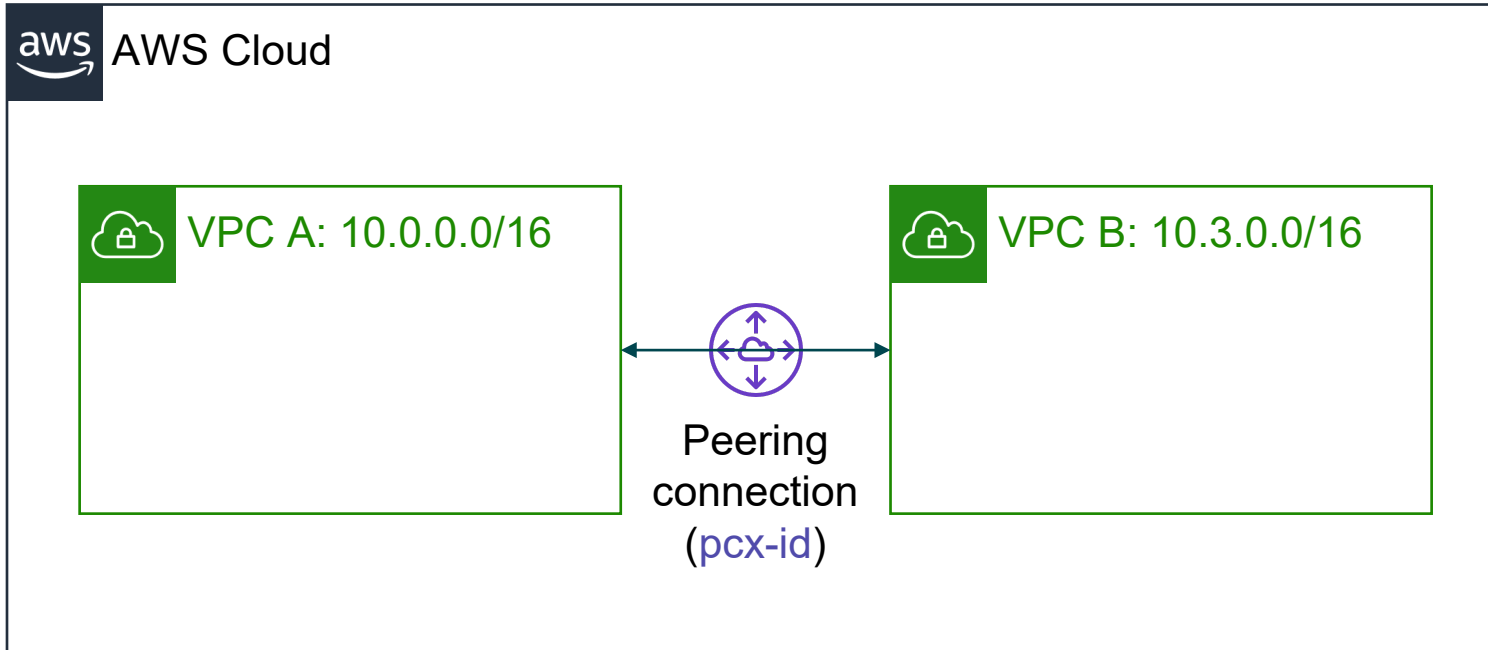
Private Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-gw-id

VPC sharing



VPC peering



Route Table for VPC

Destination ^A	Target
10.0.0.0/16	local
10.3.0.0/16	pcx-id

Route Table for VPC

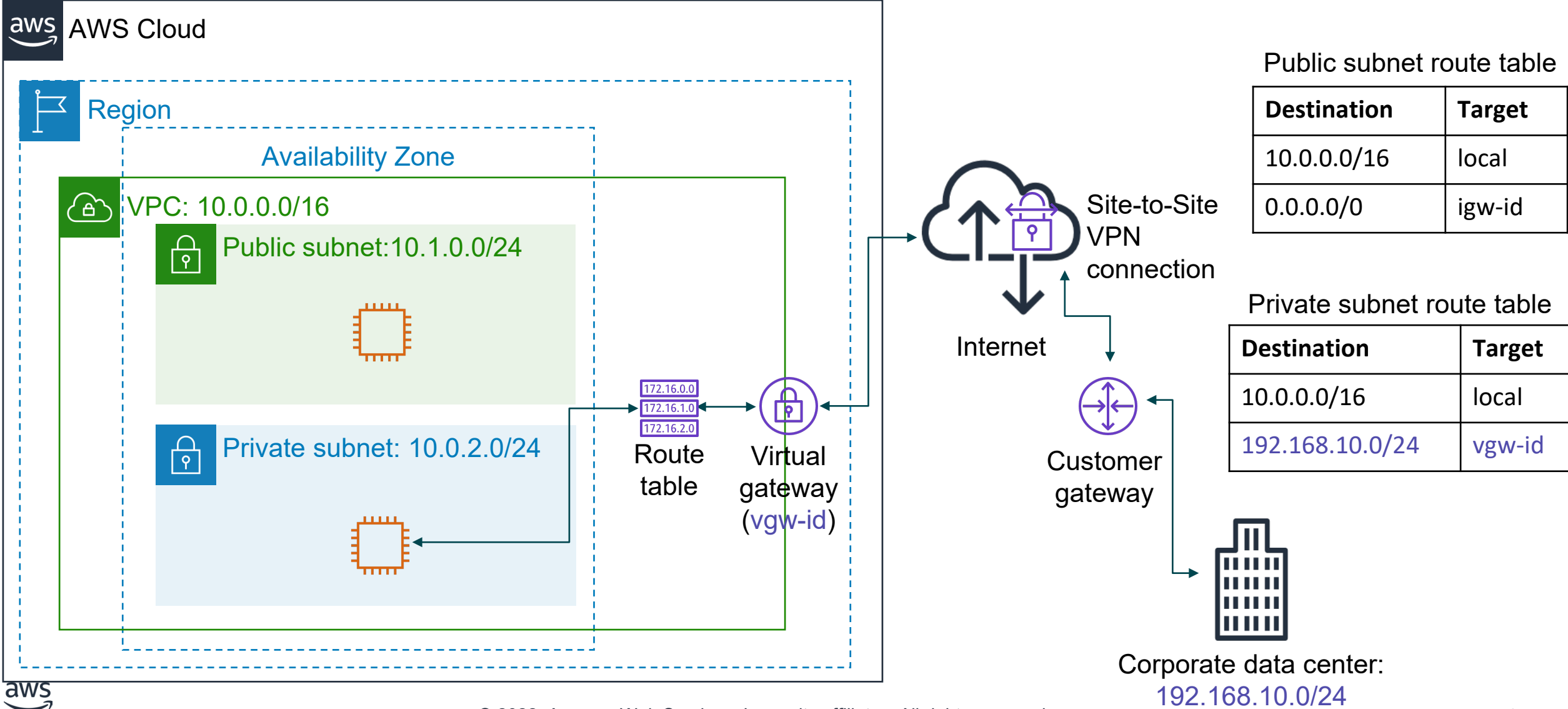
Destination ^B	Target
10.3.0.0/16	local
10.0.0.0/16	pcx-id

You can connect VPCs in your own AWS account, between AWS accounts, or between AWS Regions.

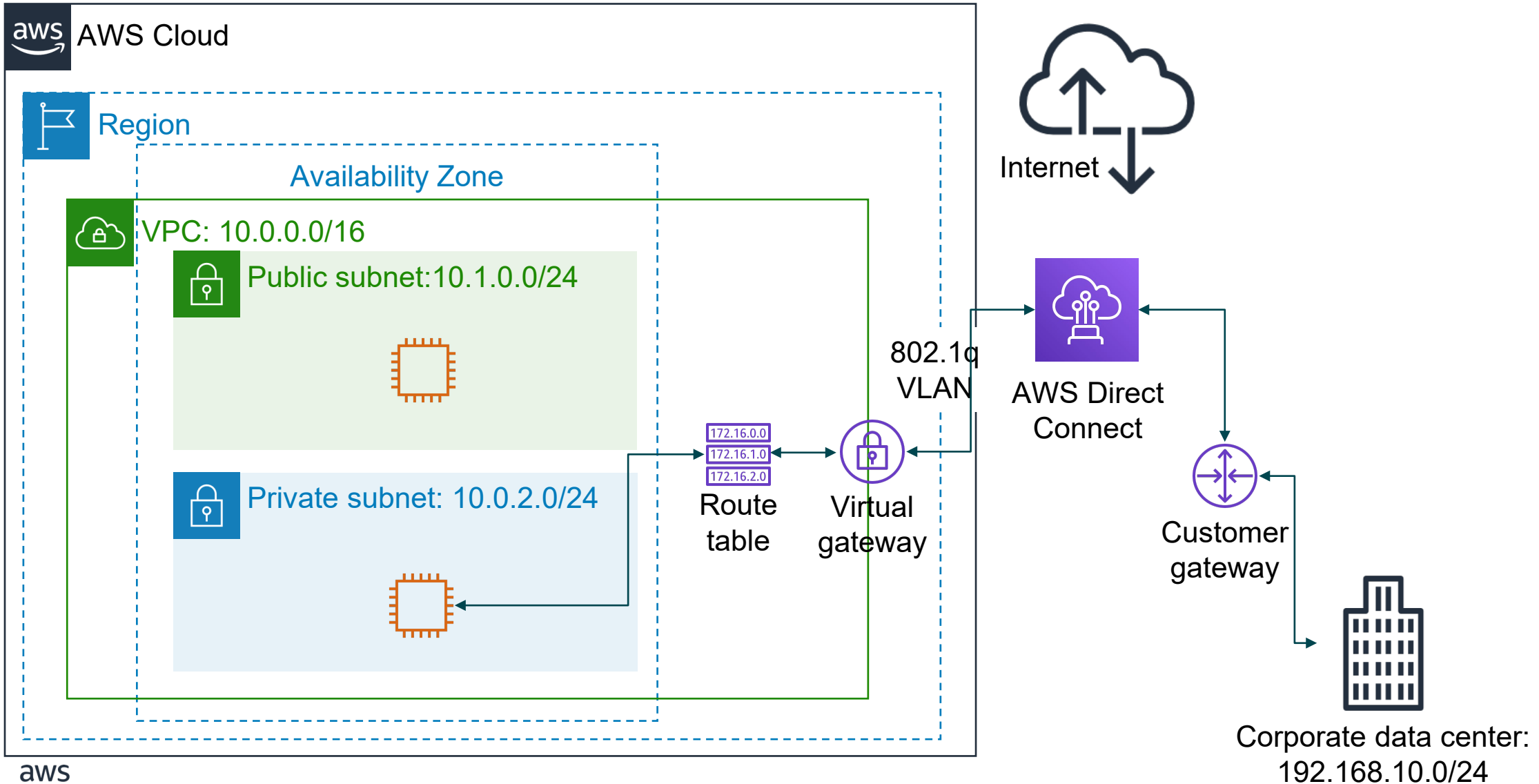
Restrictions:

- IP spaces cannot overlap.
- Transitive peering is not supported.
- You can only have one peering resource between the same two VPCs.

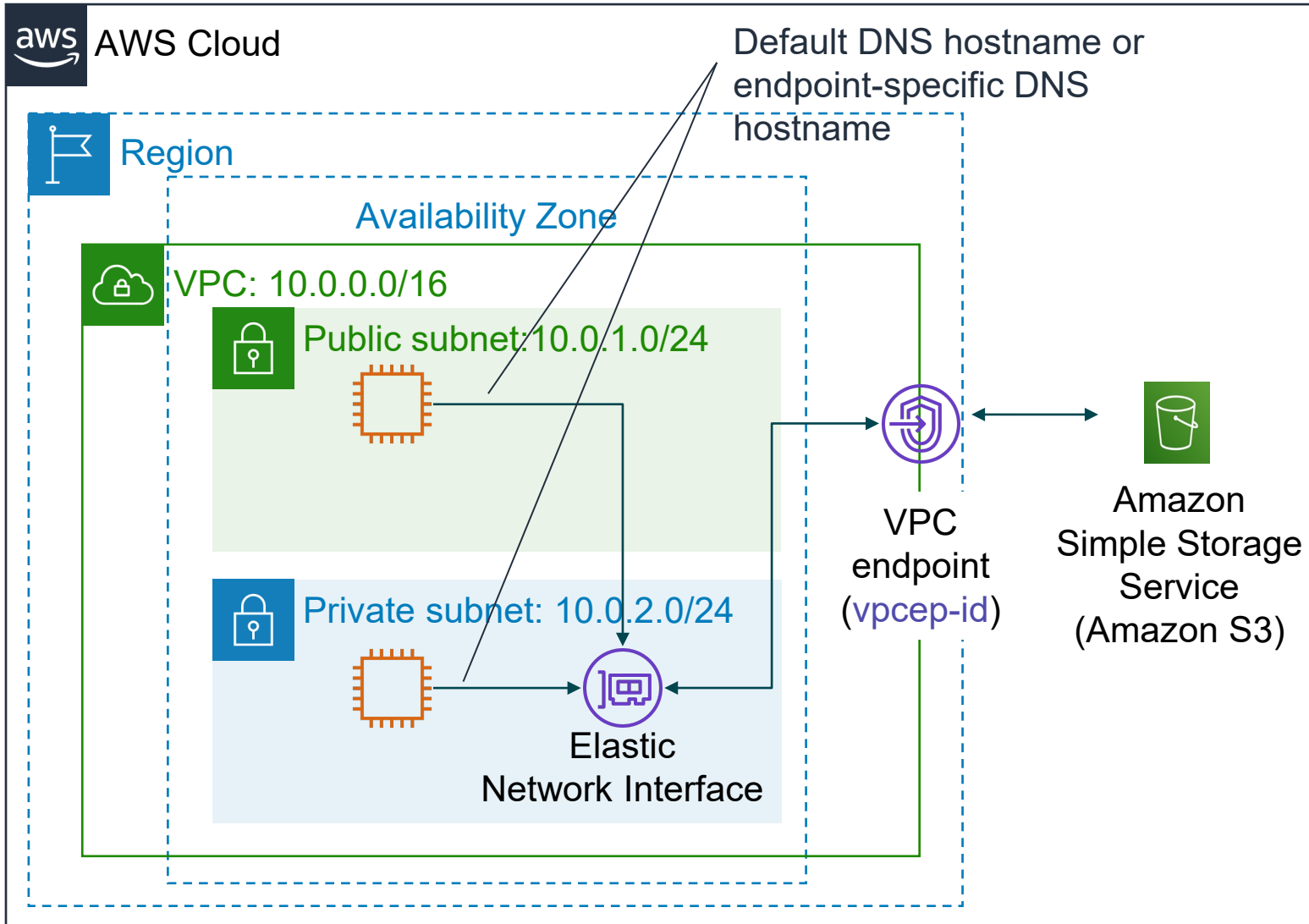
AWS Site-to-Site VPN



AWS Direct Connect



VPC endpoints



Public Subnet Route Table

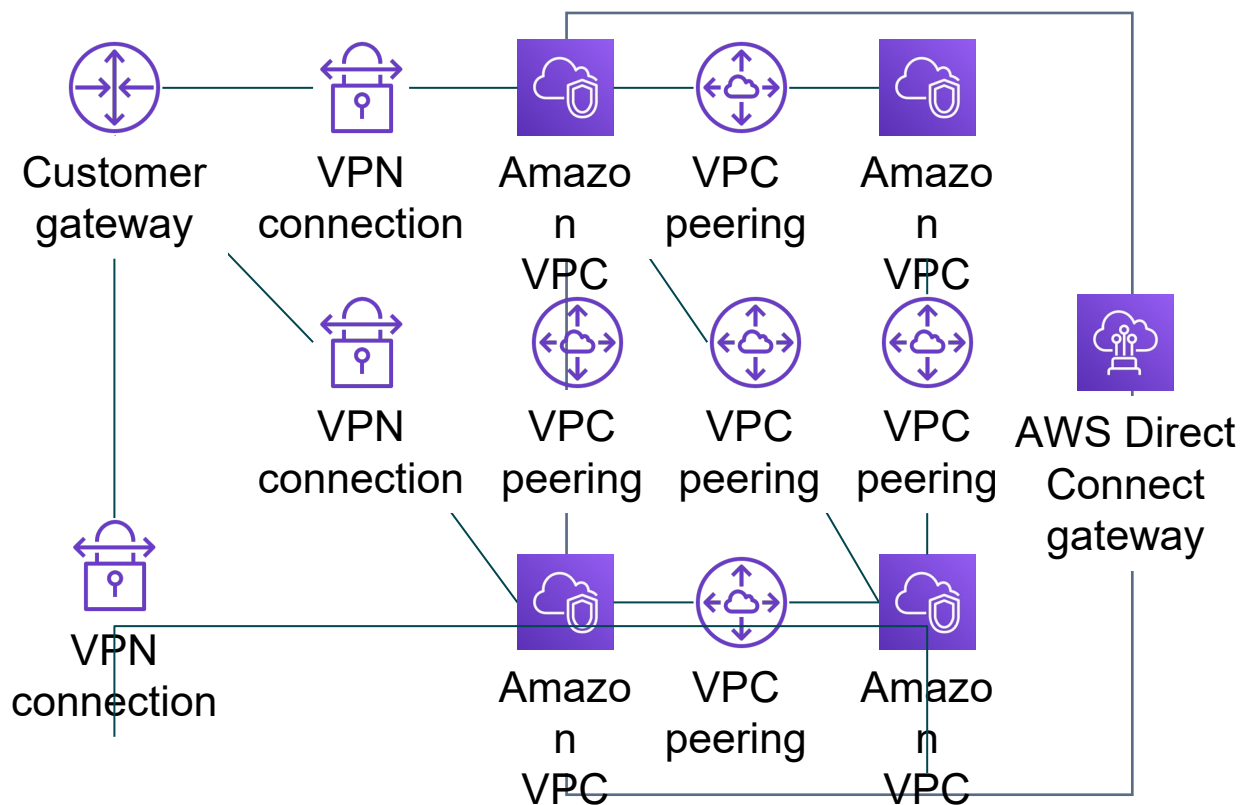
Destination	Target
10.0.0.0/16	local
Amazon S3 ID	vpcep-id

Two types of endpoints:

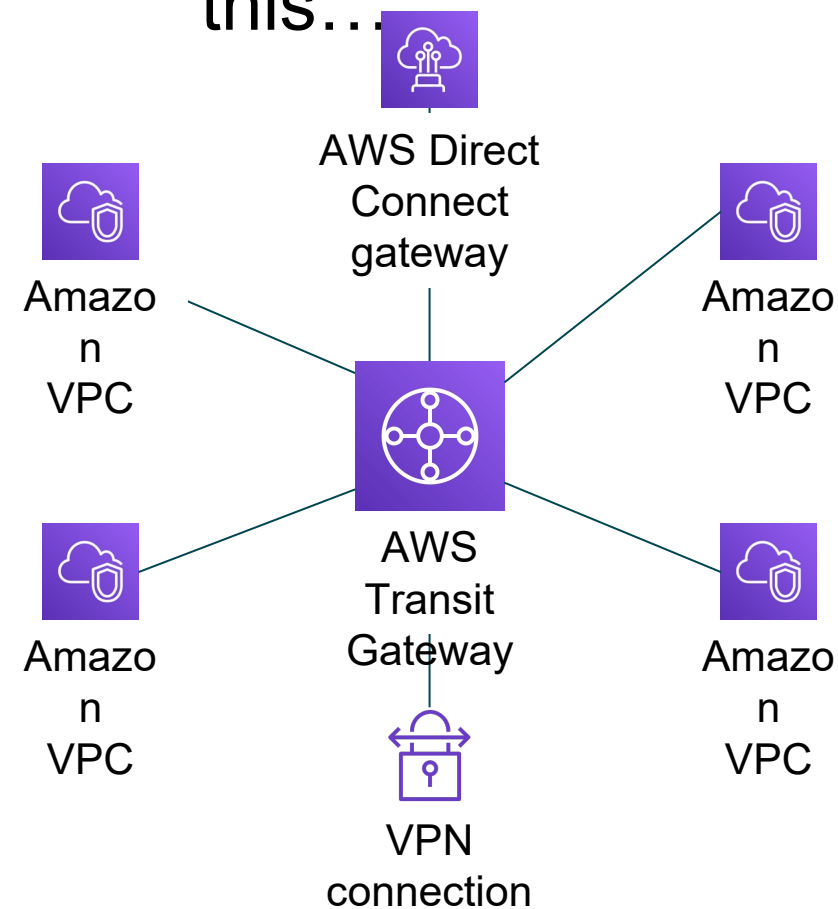
- **Interface** endpoints (powered by AWS PrivateLink)
- **Gateway** endpoints (Amazon S3 and Amazon DynamoDB)

AWS Transit Gateway

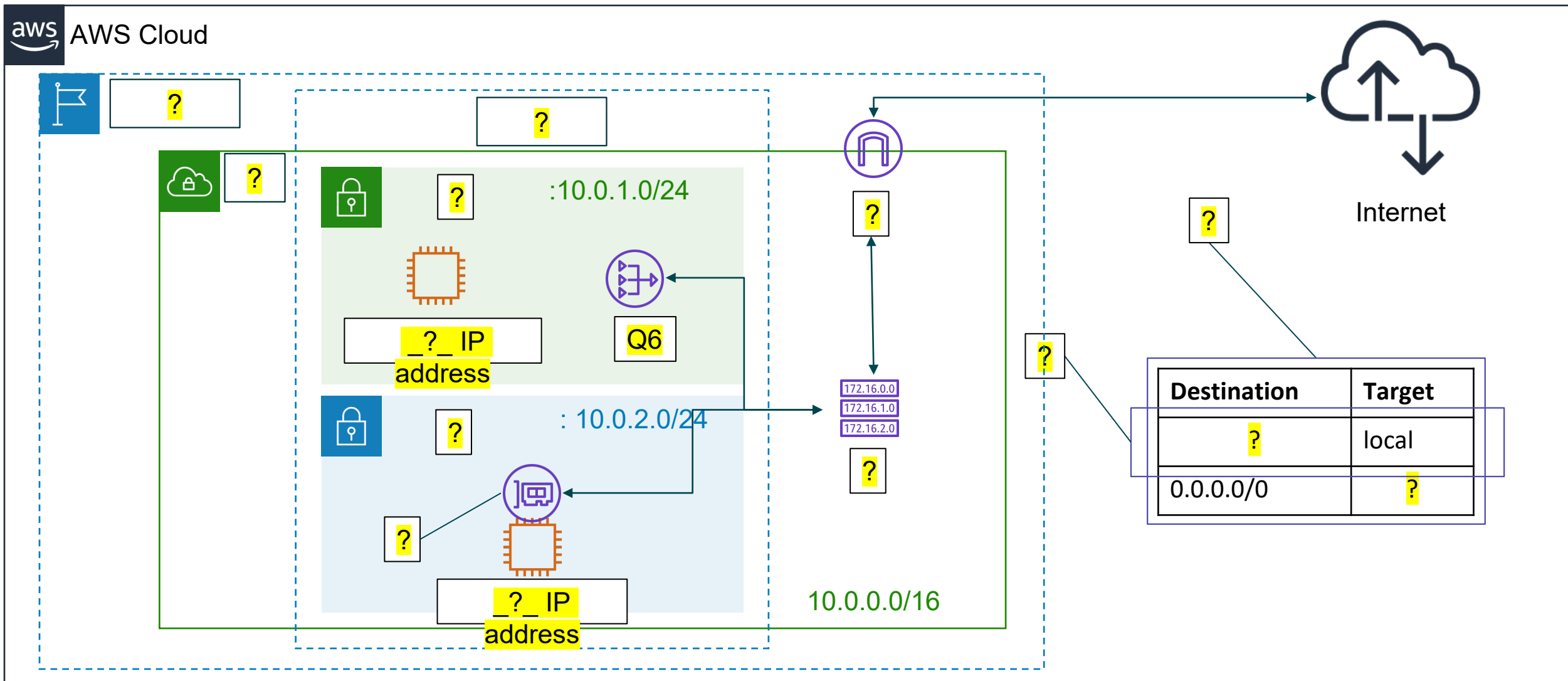
From this...



To this...



Activity: Label this network diagram



Recorded Amazon VPC demonstration



Section 3 key takeaways



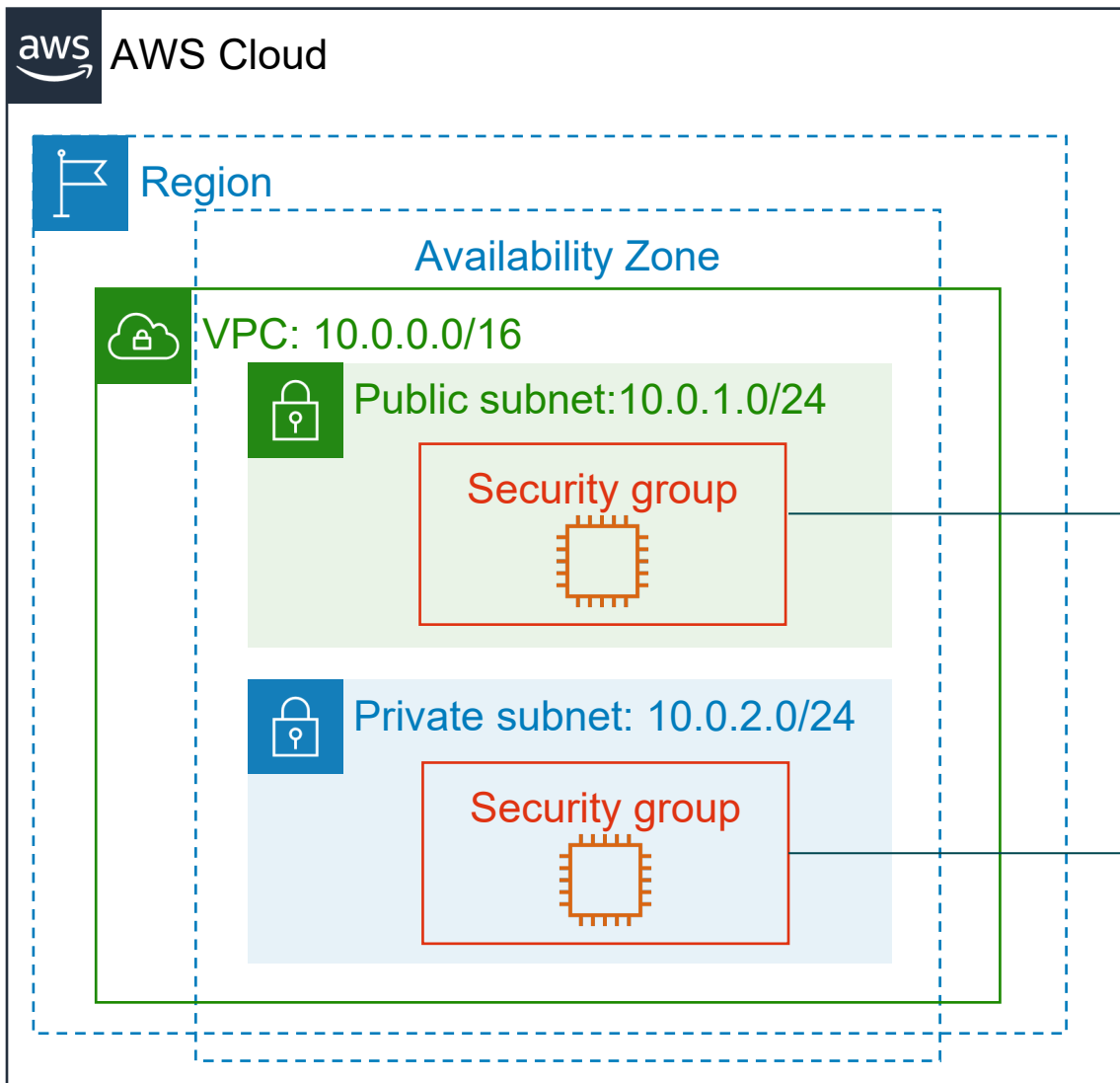
- There are several VPC networking options, which include:
 - Internet gateway
 - NAT gateway
 - VPC endpoint
 - VPC peering
 - VPC sharing
 - AWS Site-to-Site VPN
 - AWS Direct Connect
 - AWS Transit Gateway
- You can use the VPC Wizard to implement your design.



Section 4: VPC security

Module 5: Networking and Content Delivery

Security groups (1 of 2)



Security groups act at the **instance level**.



Security groups (2 of 2)

- Security groups have **rules** that control inbound and outbound instance traffic.
- Default security groups **deny all inbound** traffic and **allow all outbound** traffic.
- Security groups are **stateful**.

Inbound			
Source	Protocol	Port Range	Description
sg-xxxxxxx	All	All	Allow inbound traffic from network interfaces assigned to the same security group.

Outbound			
Destination	Protocol	Port Range	Description
0.0.0.0/0	All	All	Allow all outbound IPv4 traffic.
::/0	All	All	Allow all outbound IPv6 traffic.



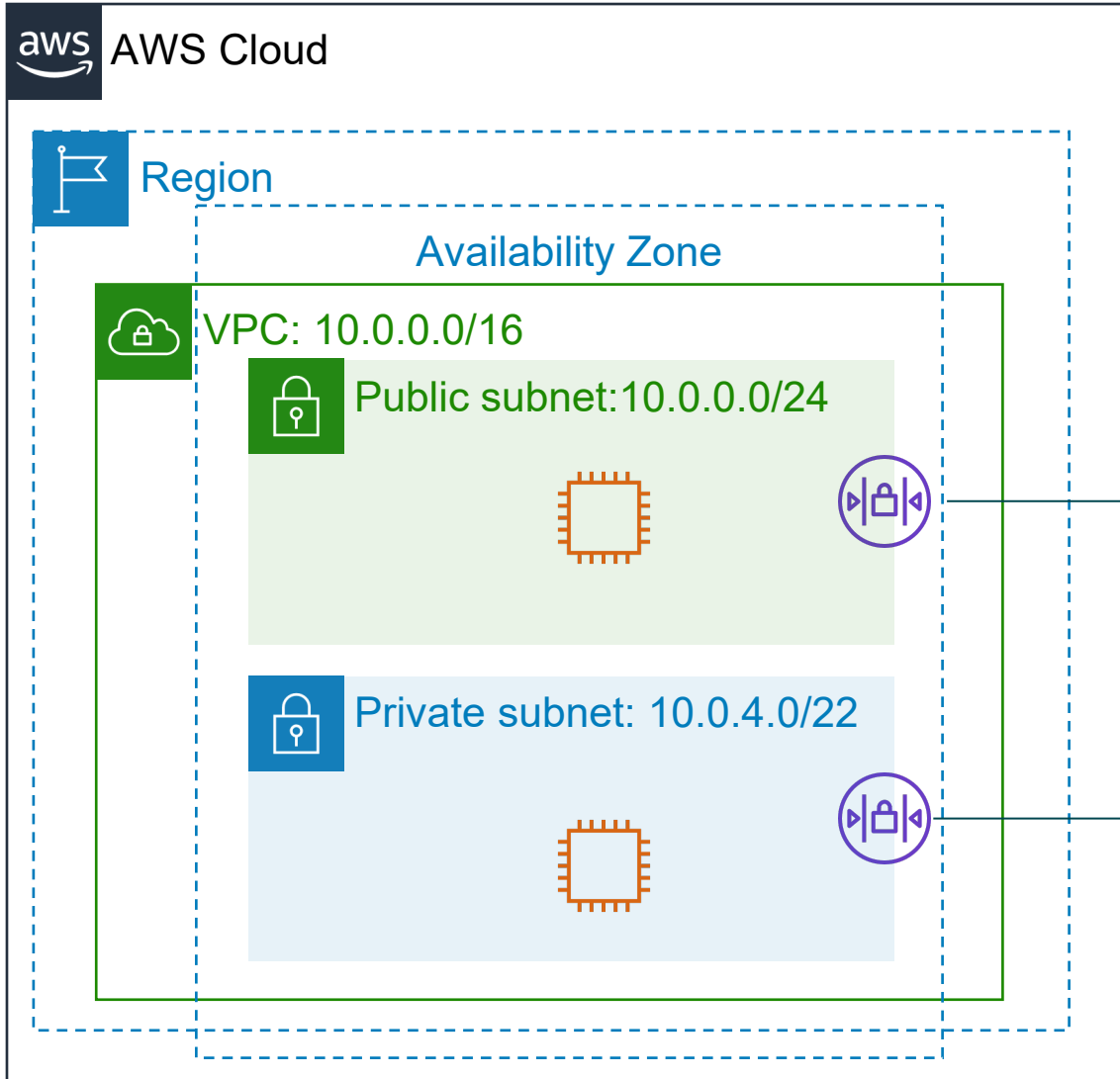
Custom security group examples

- You can **specify allow** rules, but not deny rules.
- **All rules are evaluated** before the decision to allow traffic.

Inbound			
Source	Protocol	Port Range	Description
0.0.0.0/0	TCP	80	Allow inbound HTTP access from all IPv4 addresses
0.0.0.0/0	TCP	443	Allow inbound HTTPS access from all IPv4 addresses
Your network's public IPv4 address range	TCP	22	Allow inbound SSH access to Linux instances from IPv4 IP addresses in your network (over the internet gateway)

Outbound			
Destination	Protocol	Port Range	Description
The ID of the security group for your Microsoft SQL Server database servers	TCP	1433	Allow outbound Microsoft SQL Server access to instances in the specified security group

Network access control lists (network ACLs 1 of 2)



Network ACLs act at the **subnet level**.



Network access control lists (network ACLs 2 of 2)

- A network ACL has **separate inbound and outbound rules**, and each rule can either **allow or deny traffic**.
- **Default** network ACLs **allow** all inbound and outbound IPv4 traffic.
- Network ACLs are **stateless**.

Inbound					
Rule	Type	Protocol	Port Range	Source	Allow/Deny
100	All IPv4 traffic	All	All	0.0.0.0/0	ALLOW
*	All IPv4 traffic	All	All	0.0.0.0/0	DENY

Outbound					
Rule	Type	Protocol	Port Range	Destination	Allow/Deny
100	All IPv4 traffic	All	All	0.0.0.0/0	ALLOW
*	All IPv4 traffic	All	All	0.0.0.0/0	DENY

Custom network ACLs examples

- **Custom** network ACLs **deny** all inbound and outbound traffic until you add rules.
- You can specify **both allow and deny** rules.
- Rules are evaluated in number order, starting with the **lowest number**.

Inbound

Rule	Type	Protocol	Port Range	Source	Allow/Deny
100	HTTPS	TCP	443	0.0.0.0/0	ALLOW
120	SSH	TCP	22	192.0.2.0/24	ALLOW
*	All IPv4 traffic	All	All	0.0.0.0/0	DENY

Outbound

Rule	Type	Protocol	Port Range	Destination	Allow/Deny
100	HTTPS	TCP	443	0.0.0.0/0	ALLOW
120	SSH	TCP	22	192.0.2.0/24	ALLOW
*	All IPv4 traffic	All	All	0.0.0.0/0	DENY



Security groups versus network ACLs

Attribute	Security Groups	Network ACLs
Scope	Instance level	Subnet level
Supported Rules	Allow rules only	Allow and deny rules
State	Stateful (return traffic is automatically allowed, regardless of rules)	Stateless (return traffic must be explicitly allowed by rules)
Order of Rules	All rules are evaluated before decision to allow traffic	Rules are evaluated in number order before decision to allow traffic

Section 4 key takeaways



- Build security into your VPC architecture:
 - Isolate subnets if possible.
 - Choose the appropriate gateway device or VPN connection for your needs.
 - Use firewalls.
- Security groups and network ACLs are firewall options that you can use to secure your VPC.

Lab 2: Build Your VPC and Launch a Web Server



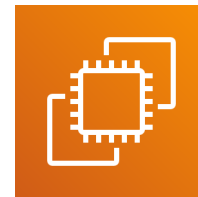


Lab 2: Scenario

In this lab, you use Amazon VPC to **create your own VPC** and add some components to produce a customized network. You **create a security group** for your VPC. You also **create an EC2 instance and configure it** to run a web server and to use the security group. You then launch the EC2 instance into the VPC.



Amazon
VPC



Amazon
EC2



Lab 2: Tasks



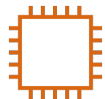
- Create a VPC.



- Create additional subnets.

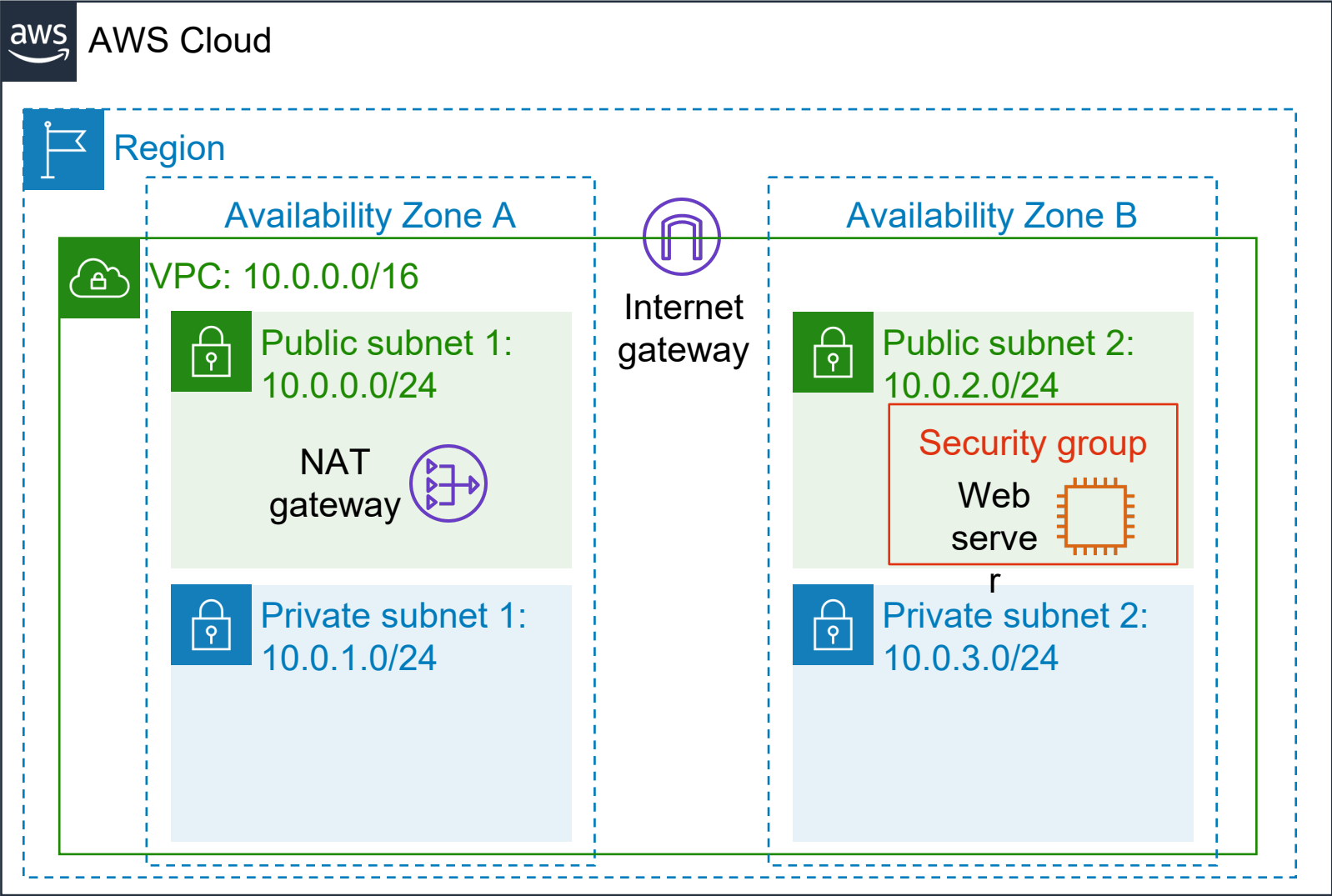
Security
group

- Create a VPC security group.



- Launch a web server instance.

Lab 2: Final product



Public Route

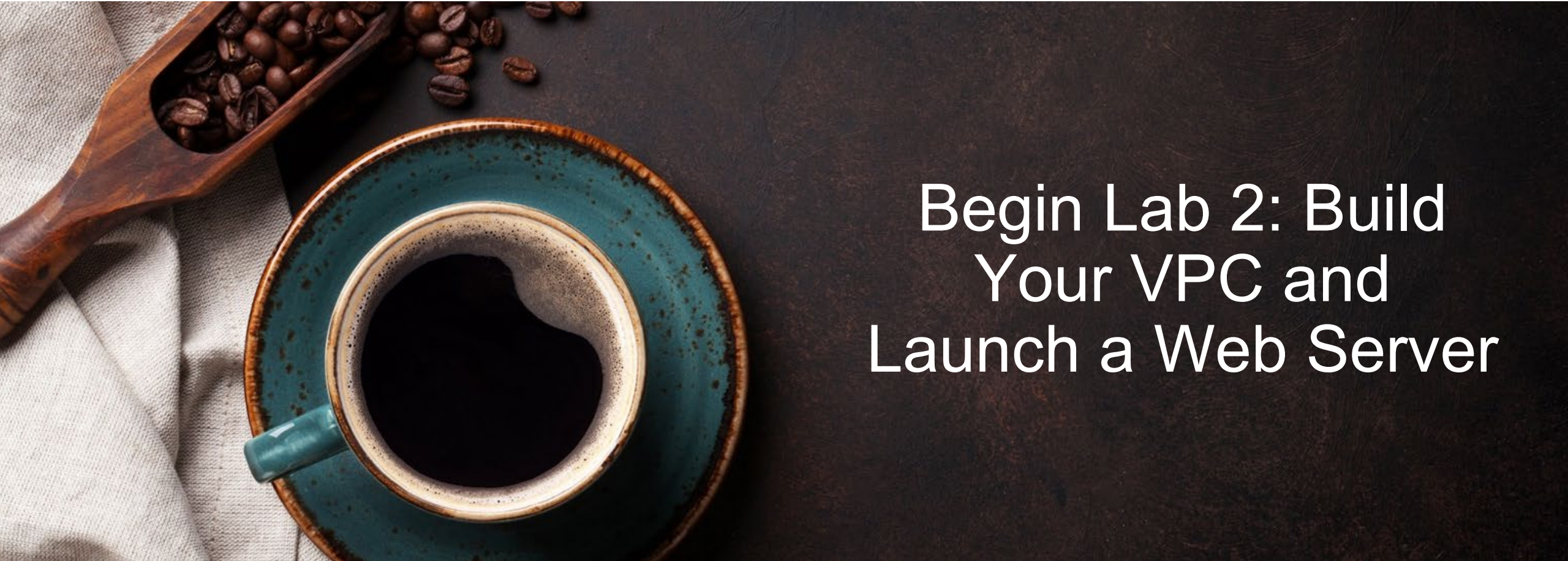
Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	Internet gateway

Private Route

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	NAT gateway



~ 30 minutes



Begin Lab 2: Build Your VPC and Launch a Web Server

Lab debrief: Key takeaways

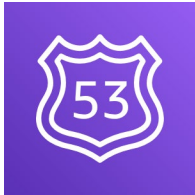




Section 5: Amazon Route 53

Module 5: Networking and Content Delivery

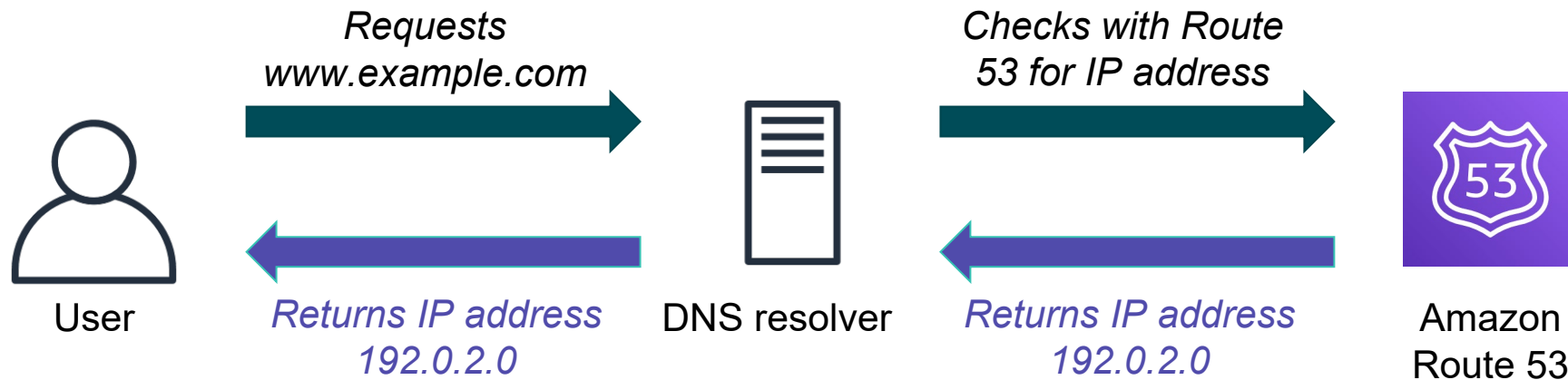
Amazon Route 53



Amazon
Route 53

- Is a highly available and scalable Domain Name System (DNS) web service
- Is used to route end users to internet applications by translating names (like www.example.com) into numeric IP addresses (like *192.0.2.1*) that computers use to connect to each other
- Is fully compliant with IPv4 and IPv6
- Connects user requests to infrastructure running in AWS and also outside of AWS
- Is used to check the health of your resources
- Features traffic flow
- Enables you to register domain names

Amazon Route 53 DNS resolution

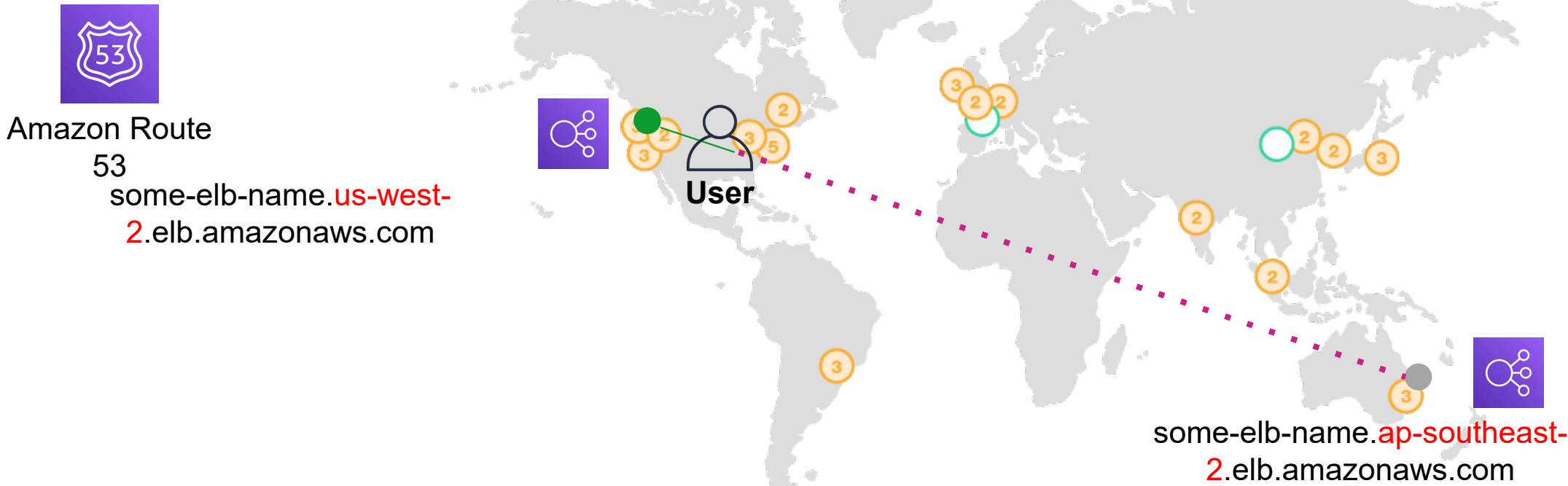




Amazon Route 53 supported routing

- **Simple routing** – Use in single-server environments
- **Weighted round robin routing** – Assign weights to resource record sets to specify the frequency
- **Latency routing** – Help improve your global applications
- **Geolocation routing** – Route traffic based on location of your users
- **Geoproximity routing** – Route traffic based on location of your resources
- **Failover routing** – Fail over to a backup site if your primary site becomes unreachable
- **Multivalue answer routing** – Respond to DNS queries with up to eight healthy records selected at random

Use case: Multi-region deployment



Name	Type	Value
example.com	ALIAS	some-elb-name.us-west-2.elb.amazonaws.com
example.com	ALIAS	some-elb-name.ap-southeast-2.elb.amazonaws.com



Amazon Route 53 DNS failover

Improve the availability of your applications that run on AWS by:

- Configuring backup and failover scenarios for your own applications
- Enabling highly available multi-region architectures on AWS
- Creating health checks

▼ Advanced configuration

Request interval ☒ Standard (30 seconds) ☐ Fast (10 seconds) ⓘ

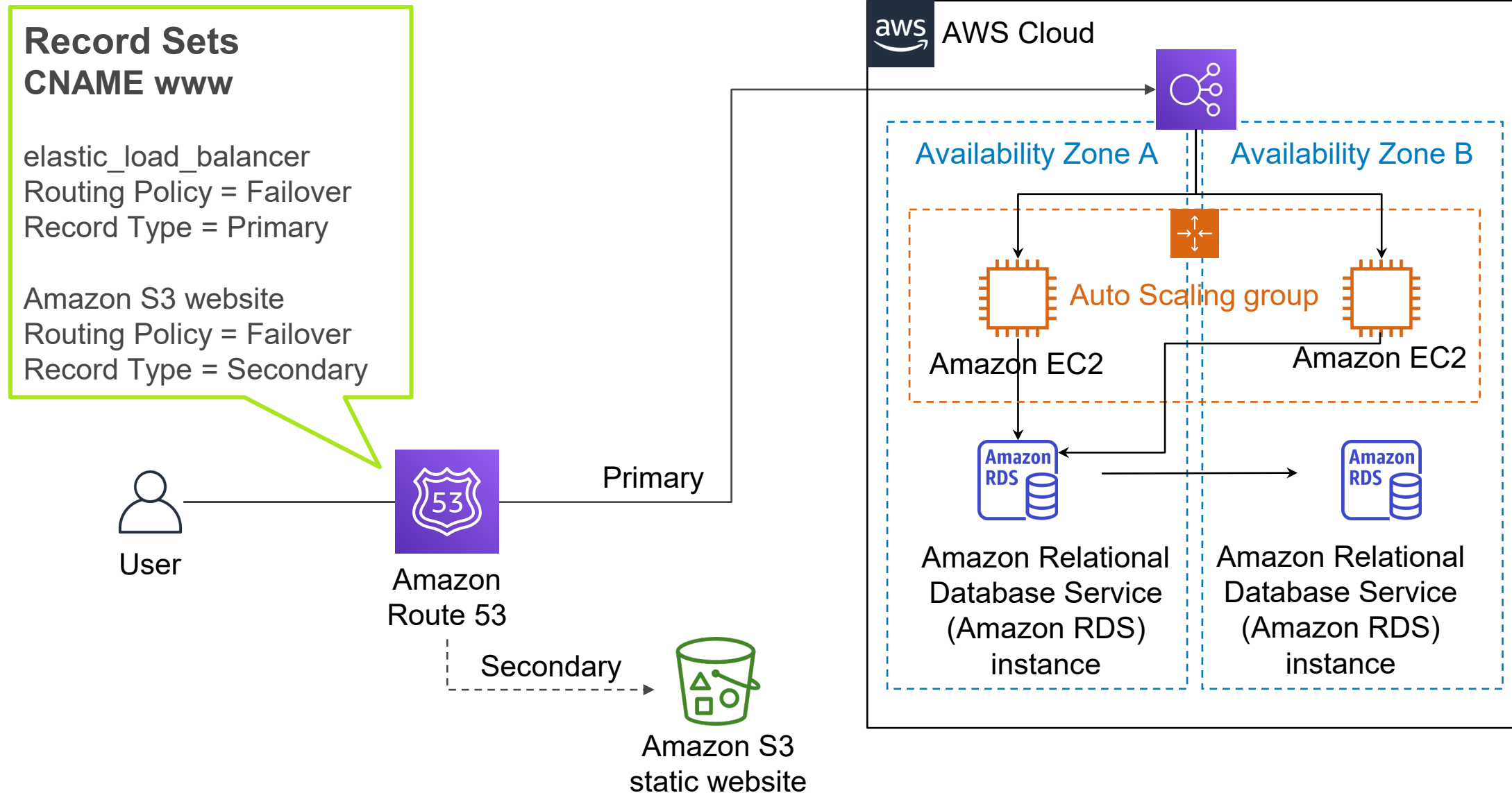
Failure threshold * ⓘ

String matching ☒ No ☐ Yes ⓘ

Latency graphs ☒ ⓘ

Invert health check status ☐ ⓘ

DNS failover for a multi-tiered web application



Section 5 key takeaways



- Amazon Route 53 is a highly available and scalable cloud DNS web service that translates domain names into numeric IP addresses.
- Amazon Route 53 supports several types of routing policies.
- Multi-Region deployment improves your application's performance for a global audience.
- You can use Amazon Route 53 failover to improve the availability of your applications.

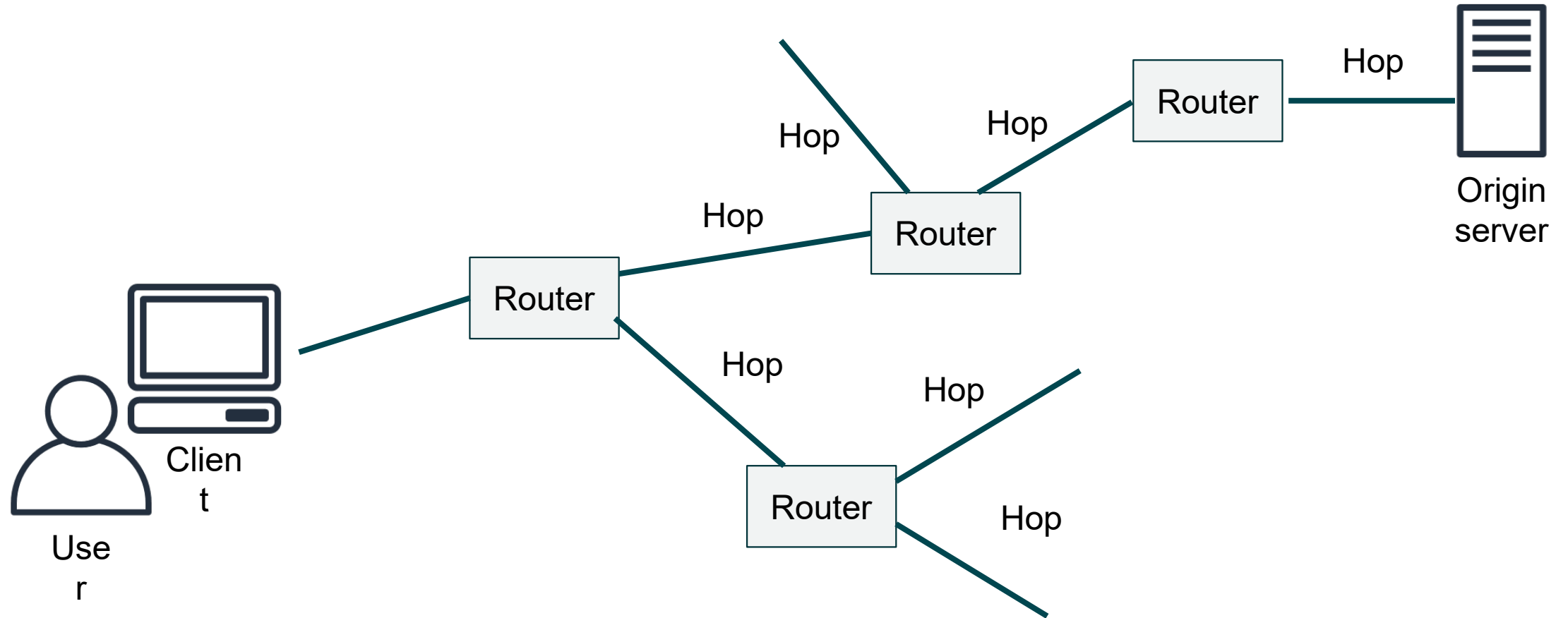


Section 6: Amazon CloudFront

Module 5: Networking and Content Delivery



Content delivery and network latency





Content delivery network (CDN)

- Is a globally distributed system of caching servers
- Caches copies of commonly requested files (static content)
- Delivers a local copy of the requested content from a nearby cache edge or Point of Presence
- Accelerates delivery of dynamic content
- Improves application performance and scaling



Amazon CloudFront



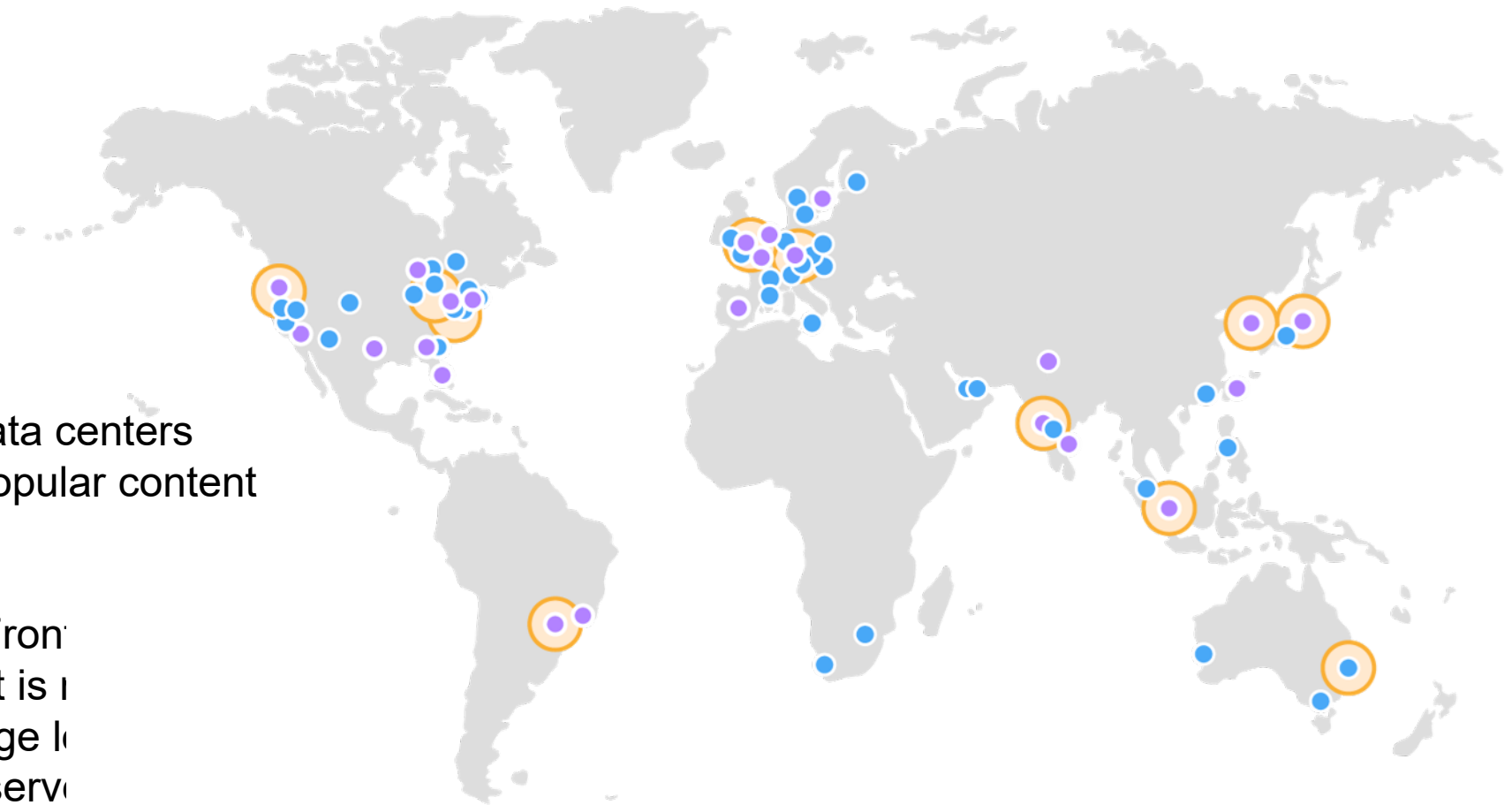
Amazon
CloudFront

- Fast, global, and secure CDN service
- Global network of edge locations and Regional edge caches
- Self-service model
- Pay-as-you-go pricing

Amazon CloudFront infrastructure

- Edge locations
- Multiple edge locations
- Regional edge caches

- **Edge locations** – Network of data centers that CloudFront uses to serve popular content quickly to customers.
- **Regional edge cache** – CloudFront location that caches content that is popular enough to stay at an edge location. It is located between the origin server and the global edge location.





Amazon CloudFront benefits

- Fast and global
- Security at the edge
- Highly programmable
- Deeply integrated with AWS
- Cost-effective



Amazon CloudFront pricing

Data transfer out

- Charged for the volume of data transferred out from Amazon CloudFront edge location to the internet or to your origin.

HTTP(S) requests

- Charged for number of HTTP(S) requests.

Invalidation requests

- No additional charge for the first 1,000 paths that are requested for invalidation each month. Thereafter, \$0.005 per path that is requested for invalidation.

Dedicated IP custom SSL

- \$600 per month for each custom SSL certificate that is associated with one or more CloudFront distributions that use the Dedicated IP version of custom SSL certificate support.

Section 6 key takeaways



- A CDN is a globally distributed system of caching servers that accelerates delivery of content.
- Amazon CloudFront is a fast CDN service that securely delivers data, videos, applications, and APIs over a global infrastructure with low latency and high transfer speeds.
- Amazon CloudFront offers many benefits.



Module wrap-up

Module 5: Networking and Content Delivery



Module summary

In summary, in this module you learned how to:

- Recognize the basics of networking
- Describe virtual networking in the cloud with Amazon VPC
- Label a network diagram
- Design a basic VPC architecture
- Indicate the steps to build a VPC
- Identify security groups
- Create your own VPC and added additional components to it to produce a customized network
- Identify the fundamentals of Amazon Route 53

Complete the knowledge check





Sample exam question

Which AWS networking service enables a company to create a virtual network within AWS?

Choice	Response
A	AWS Config
B	Amazon Route 53
C	AWS Direct Connect
D	Amazon VPC



Additional resources

- Amazon VPC Overview pag:
<https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>
- Amazon Virtual Private Cloud Connectivity Options whitepaper:
<https://docs.aws.amazon.com/whitepapers/latest/aws-vpc-connectivity-options/introduction.html>
- One to Many: Evolving VPC Design AWS Architecture blog post:
<https://aws.amazon.com/blogs/architecture/one-to-many-evolving-vpc-design/>
- Amazon VPC User Guide:
<https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>
- Amazon CloudFront overview page:
<https://aws.amazon.com/cloudfront/?nc=sn&loc=1>



Module 6: Compute

AWS Academy Cloud Foundations



Module overview

Topics

- Compute services overview
- Amazon EC2
- Amazon EC2 cost optimization
- Container services
- Introduction to AWS Lambda
- Introduction to AWS Elastic Beanstalk

Activities

- Amazon EC2 versus Managed Service
- Hands-on with AWS Lambda
- Hands-on with AWS Elastic Beanstalk

Demo

- Recorded demonstration of Amazon EC2

Lab

- Introduction to Amazon EC2



**Knowledge
check**



Module objectives

After completing this module, you should be able to:

- Provide an overview of different AWS compute services in the cloud
- Demonstrate why to use Amazon Elastic Compute Cloud (Amazon EC2)
- Identify the functionality in the EC2 console
- Perform basic functions in Amazon EC2 to build a virtual computing environment
- Identify Amazon EC2 cost optimization elements
- Demonstrate when to use AWS Elastic Beanstalk
- Demonstrate when to use AWS Lambda
- Identify how to run containerized applications in a cluster of managed servers



Section 1: Compute services overview

Module 6: Compute

AWS compute services

Amazon Web Services (AWS) offers many compute services. This module will discuss the highlighted services.



Amazon EC2



Amazon EC2
Auto Scaling



Amazon Elastic
Container Registry
(Amazon ECR)



Amazon Elastic
Container Service
(Amazon ECS)



VMware Cloud
on AWS



AWS Elastic
Beanstalk



AWS Lambda



Amazon Elastic
Kubernetes Service
(Amazon EKS)



Amazon Lightsail



AWS Batch



AWS Fargate



AWS Outposts



AWS Serverless
Application Repository

Categorizing compute services

Services	Key Concepts	Characteristics	Ease of Use
Amazon EC2	- Infrastructure as a service (IaaS) - Instance-based Virtual machines	Provision virtual machines that you can manage as you choose	A familiar concept to many IT professionals.
AWS Lambda	Serverless computing - Function-based - Low-cost	- Write and deploy code that runs on a schedule or that can be triggered by events - Use when possible (architect for the cloud)	A relatively new concept for many IT staff members, but easy to use after you learn how.
- Amazon ECS - Amazon EKS - AWS Fargate - Amazon ECR	Container-based computing - Instance-based	Spin up and run jobs more quickly	AWS Fargate reduces administrative overhead, but you can use options that give you more control.
AWS Elastic Beanstalk	- Platform as a service (PaaS) - For web applications	- Focus on your code (building your application) - Can easily tie into other services—databases, Domain Name System (DNS), etc.	Fast and easy to get started.



Choosing the optimal compute service

- The optimal compute service or services that you use will depend on your use case
- Some aspects to consider –
 - What is your application design?
 - What are your usage patterns?
 - Which configuration settings will you want to manage?
- Selecting the wrong compute solution for an architecture can lead to lower performance efficiency
 - A good starting place—Understand the available compute options



Section 2: Amazon EC2

Module 6: Compute



Amazon Elastic Compute Cloud (Amazon EC2)

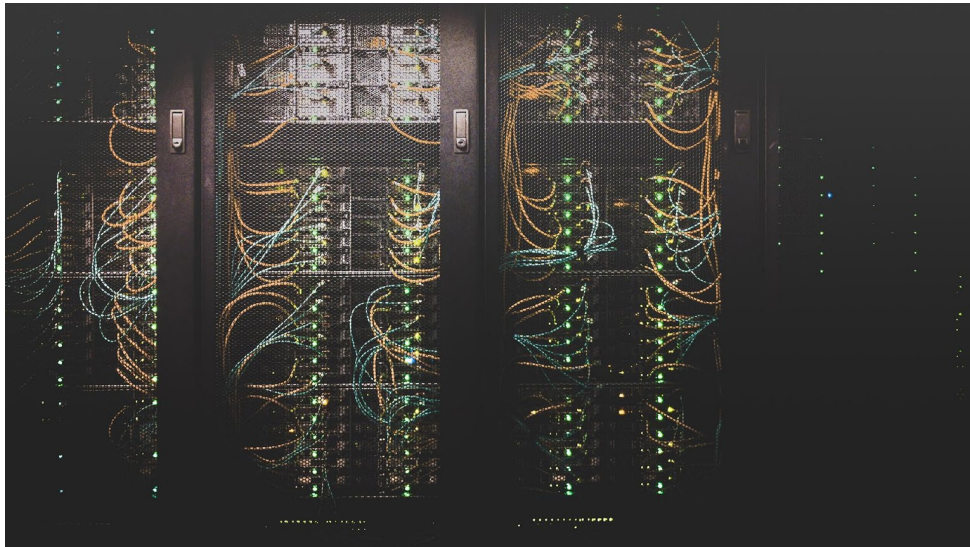
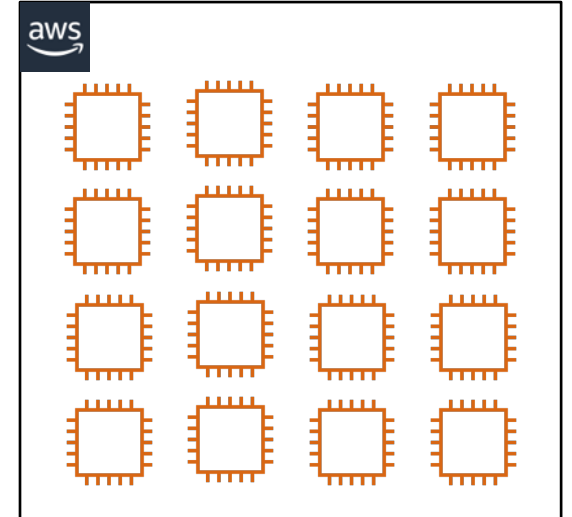


Photo by Taylor
Vick on Unsplash

**On-premises
servers**

Example uses of Amazon EC2 instances

- ✓ Application server
- ✓ Web server
- ✓ Database server
- ✓ Game server
- ✓ Mail server
- ✓ Media server
- ✓ Catalog server
- ✓ File server
- ✓ Computing server
- ✓ Proxy server



**Amazon EC2
instances**

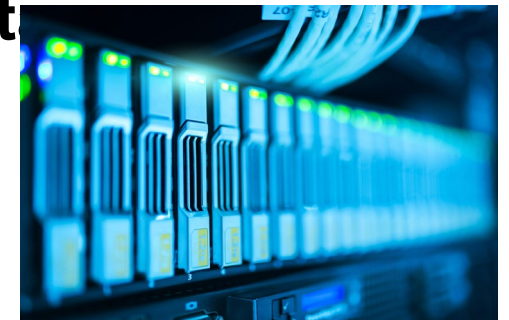
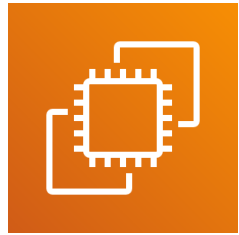


Photo by panumas nikhomkhai from Pexels

Amazon EC2 overview



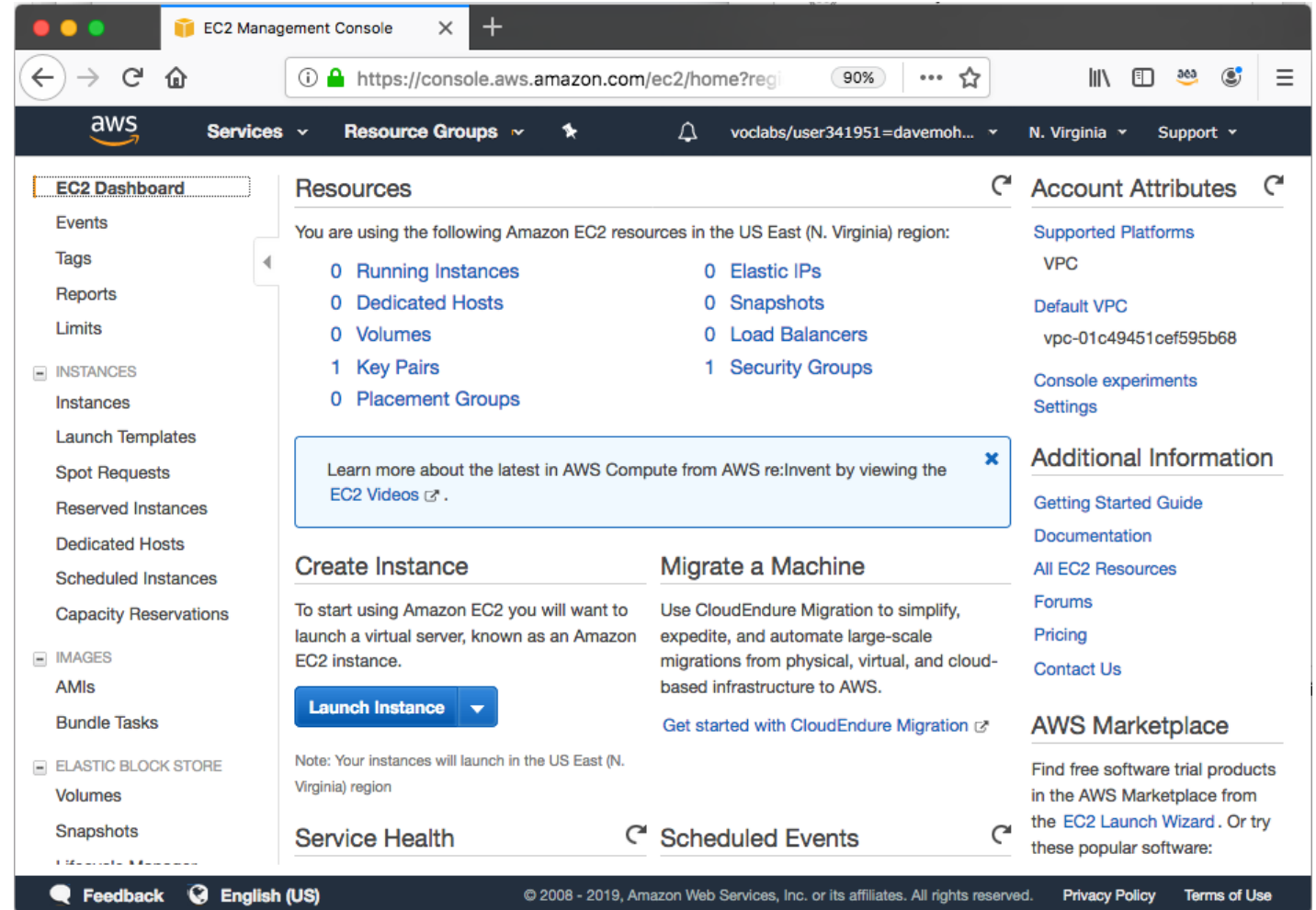
Amazon
EC2

- **Amazon Elastic Compute Cloud (Amazon EC2)**
 - Provides **virtual machines**—referred to as **EC2 instances**—in the cloud.
 - Gives you *full control* over the guest operating system (Windows or Linux) on each instance.
- You can launch instances of any size into an Availability Zone anywhere in the world.
 - Launch instances from **Amazon Machine Images (AMIs)**.
 - Launch instances with a few clicks or a line of code, and they are ready in minutes.

Launching an Amazon EC2 instance

This section of the module walks through **nine key decisions** to make when you create an EC2 instance by using the AWS Management Console **Launch Instance Wizard**.

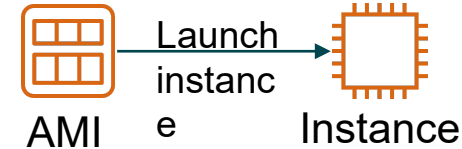
- Along the way, essential Amazon EC2 concepts will be explored.



1. Select an AMI

Choices made using the Launch Instance Wizard:

1. **AMI**
2. **Instance Type**
3. **Network settings**
4. **IAM role**
5. **User data**
6. **Storage options**
7. **Tags**
8. **Security group**
9. **Key pair**

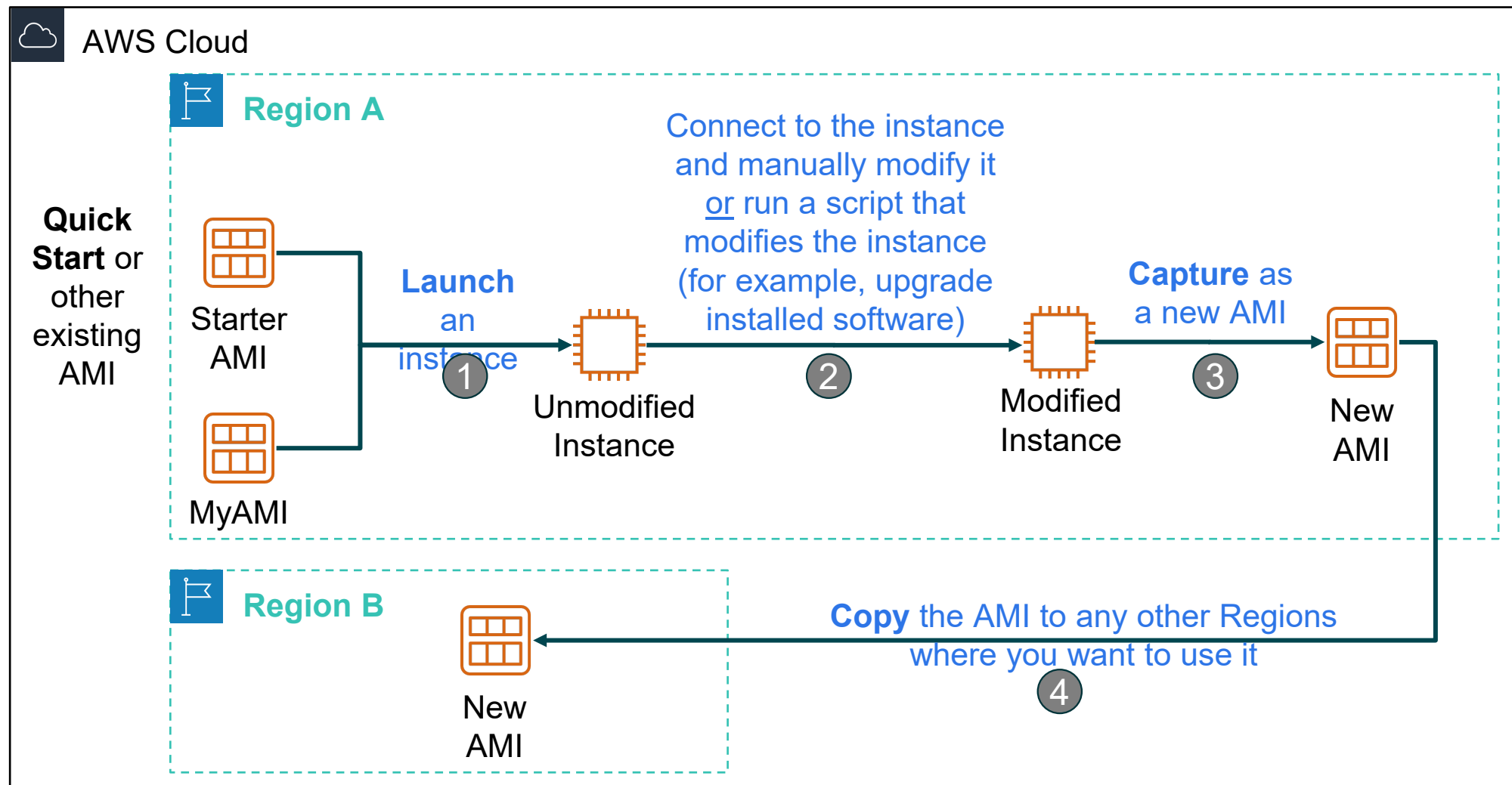


- Amazon Machine Image (AMI)
 - Is a template that is used to create an EC2 instance (which is a **virtual machine, or VM**, that runs in the AWS Cloud)
 - Contains a **Windows** or **Linux** operating system
 - Often also has some **software** pre-installed
- AMI choices:
 - Quick Start – *Linux and Windows AMIs that are provided by AWS*
 - My AMIs – *Any AMIs that you created*
 - AWS Marketplace – *Pre-configured templates from third parties*
 - Community AMIs – *AMIs shared by others; use at your own risk*

Creating a new AMI: Example

AMI details

(Optional) Import a virtual machine



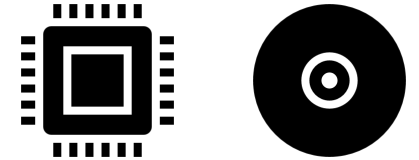


2. Select an instance type

Choices made using the Launch Instance Wizard:

1. AMI
2. **Instance Type**
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Consider your use case
 - How will the EC2 instance you create be used?
- The **instance type** that you choose determines –
 - Memory (RAM)
 - Processing power (CPU)
 - Disk space and disk type (Storage)
 - Network performance
- Instance type categories –
 - General purpose
 - Compute optimized
 - Memory optimized
 - Storage optimized
 - Accelerated computing
- Instance types offer *family*, *generation*, and *size*





EC2 instance type naming and sizes

Instance type naming

- Example: **t3.large**
 - **T** is the family name
 - **3** is the generation number
 - **Large** is the size

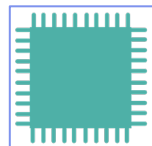
Example instance sizes

Instance Name	vCPU	Memory (GB)	Storage
t3.nano	2	0.5	EBS-Only
t3.micro	2	1	EBS-Only
t3.small	2	2	EBS-Only
t3.medium	2	4	EBS-Only
t3.large	2	8	EBS-Only
t3.xlarge	4	16	EBS-Only
t3.2xlarge	8	32	EBS-Only

Select instance type: Based on use case



**General
Purpose**



**Compute
Optimized**



**Memory
Optimized**



**Accelerated
Computing**



**Storage
Optimized**

Instance Types	a1, m4, m5, t2, t3	c4, c5	r4, r5, x1, z1	f1, g3, g4, p2, p3	d2, h1, i3
Use Case	Broad	High performance	In-memory databases	Machine learning	Distributed file systems



Instance types: Networking features

- The network bandwidth (Gbps) varies by instance type.
 - See [Amazon EC2 Instance Types](#) to compare.
- To maximize networking and bandwidth performance of your instance type:
 - If you have interdependent instances, launch them into a **cluster placement group**.
 - Enable enhanced networking.
- Enhanced networking types are supported on most instance types.
 - See the [Networking and Storage Features](#) documentation for details.
- Enhanced networking types –
 - **Elastic Network Adapter (ENA)**: Supports network speeds of up to 100 Gbps.
 - **Intel 82599 Virtual Function interface**: Supports network speeds of up to 10 Gbps.

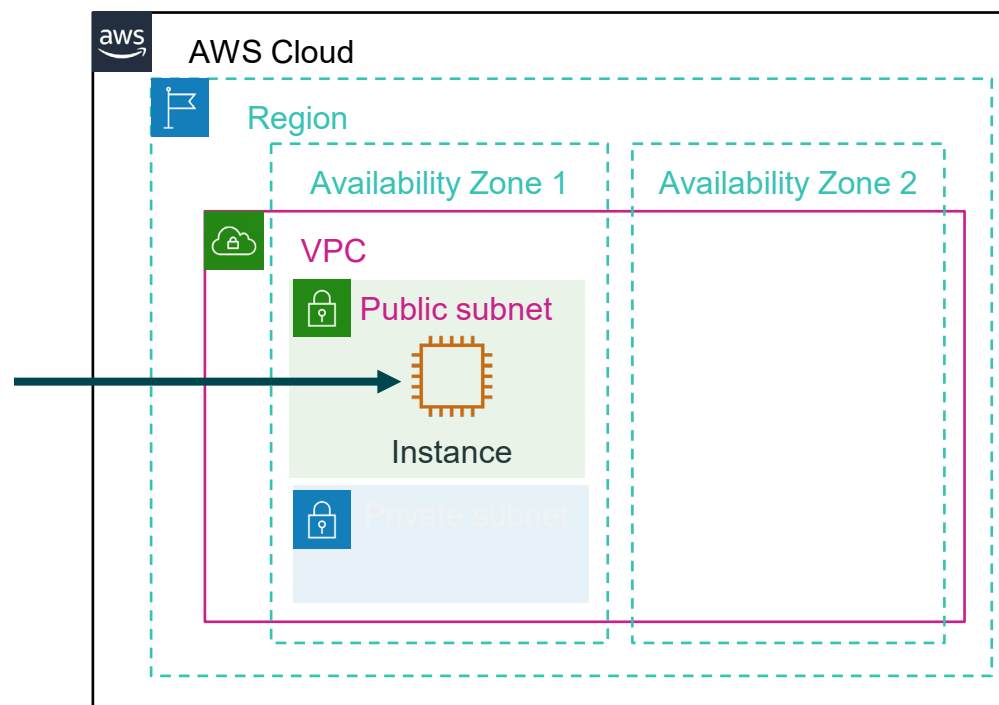
3. Specify network settings

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Where should the instance be deployed?
 - Identify the **VPC** and optionally the **subnet**
- Should a **public IP address** be automatically assigned?
 - To make it internet-accessible

*Example:
specify to
deploy the
instance here*



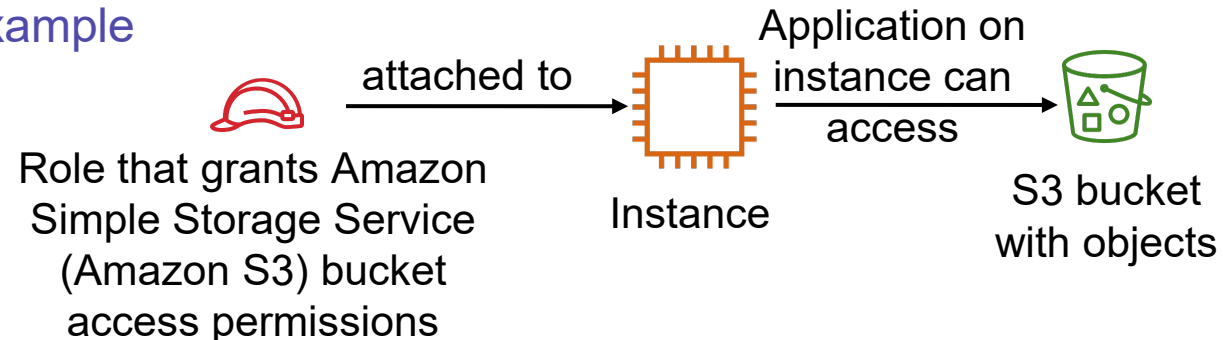
4. Attach IAM role (optional)

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. **IAM role**
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Will software on the EC2 instance need to interact with other AWS services?
- If yes, attach an appropriate **IAM Role**.
- An AWS Identity and Access Management (IAM) role that is attached to an EC2 instance is kept in an **instance profile**.
- You are *not* restricted to attaching a role only at instance launch.
- You can also attach a role to an instance that already exists.

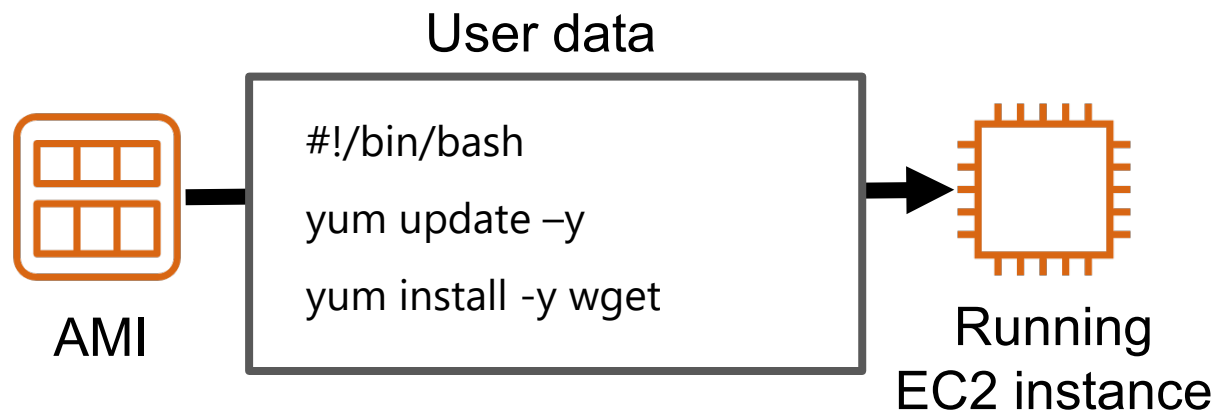
Example



5. User data script (optional)

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. **User data**
6. Storage options
7. Tags
8. Security group
9. Key pair



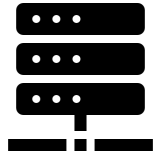
- Optionally specify a user data script at instance launch
- Use **user data** scripts to customize the runtime environment of your instance
 - Script runs the first time the instance starts
- Can be used strategically
 - For example, reduce the number of custom AMIs that you build and maintain

6. Specify storage

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Configure the **root volume**
 - Where the guest operating system is installed
- Attach **additional storage volumes** (optional)
 - AMI might already include more than one volume
- For each volume, specify:
 - The **size** of the disk (in GB)
 - The **volume type**
 - Different types of solid state drives (SSDs) and hard disk drives (HDDs) are available
 - If the volume will be deleted when the instance is terminated
 - If **encryption** should be used





Amazon EC2 storage options

- **Amazon Elastic Block Store (Amazon EBS)** –
 - Durable, block-level storage volumes.
 - You can stop the instance and start it again, and the data will still be there.
- **Amazon EC2 Instance Store** –
 - Ephemeral storage is provided on disks that are attached to the host computer where the EC2 instance is running.
 - If the instance stops, data stored here is deleted.
- Other options for storage (not for the root volume) –
 - Mount an **Amazon Elastic File System (Amazon EFS)** file system.
 - Connect to **Amazon Simple Storage Service (Amazon S3)**.

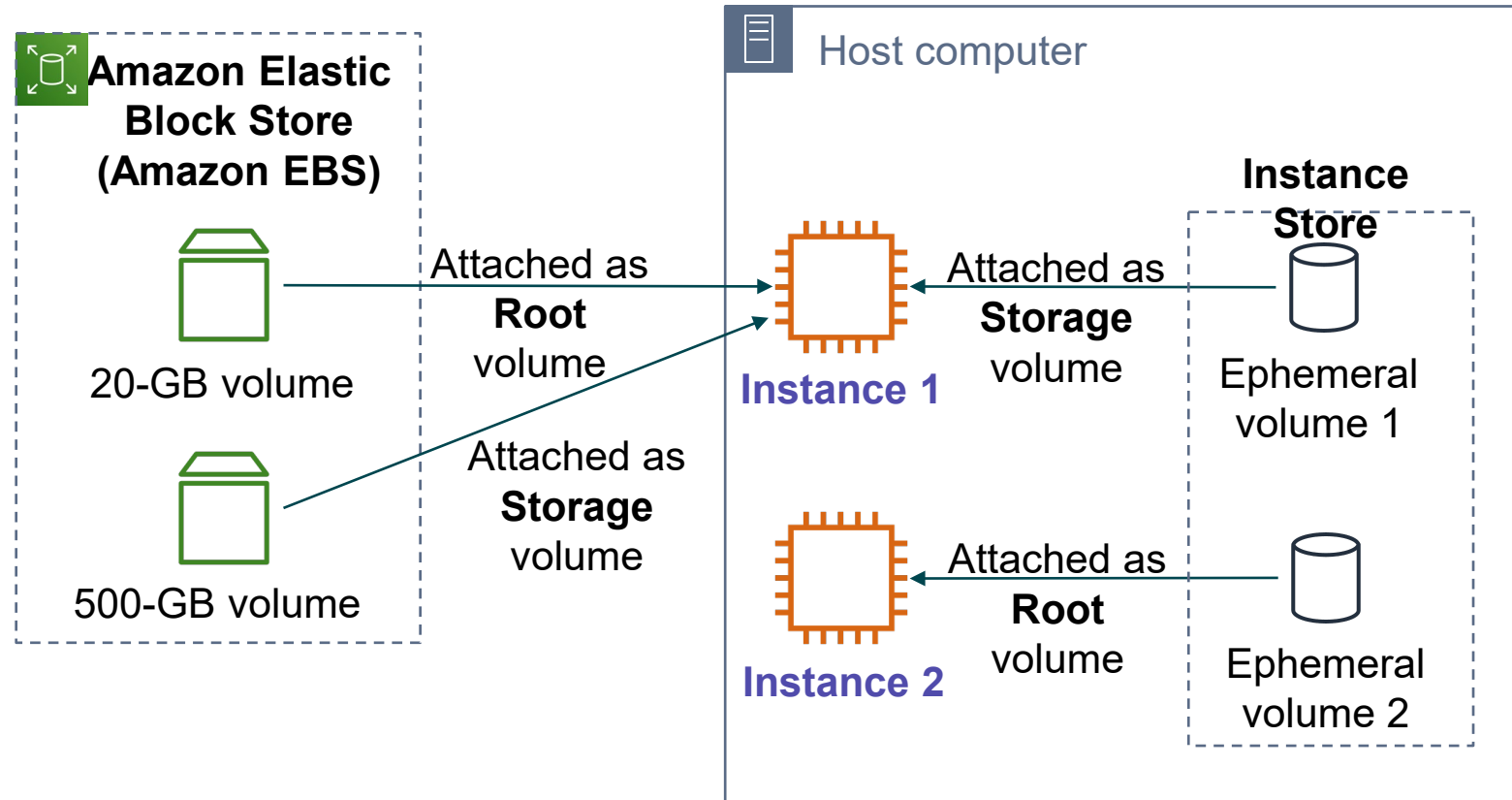
Example storage options

- **Instance 1** characteristics –

- It has an **Amazon EBS** *root volume* type for the operating system.
- What will happen if the instance is stopped and then started again?

- **Instance 2** characteristics –

- It has an **Instance Store** *root volume* type for the operating system.
- What will happen if the instance stops (because of user error or a system malfunction)?





7. Add tags

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. **Tags**
8. Security group
9. Key pair

- A **tag** is a label that you can assign to an AWS resource.
 - Consists of a *key* and an optional *value*.
- Tagging is how you can attach **metadata** to an EC2 instance.
- Potential benefits of tagging—Filtering, automation, cost allocation, and access control.

Example:

Key (128 characters maximum)	Value (256 characters maximum)
Name	WebServer1
<button>Add another tag</button> (Up to 50 tags maximum)	

8. Security group settings

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. **Security group**
9. Key pair

- A **security group** is a **set of firewall rules** that control traffic to the instance.
 - It exists *outside* of the instance's guest OS.
- Create **rules** that specify the **source** and which **ports** that network communications can use.
 - Specify the **port** number and the **protocol**, such as Transmission Control Protocol (TCP), User Datagram Protocol (UDP), or Internet Control Message Protocol (ICMP).
 - Specify the **source** (for example, an IP address or another security group) that is allowed to use the rule.

Example rule:

Type ⓘ	Protocol ⓘ	Port Range ⓘ	Source ⓘ
SSH ▾	TCP	22	My IP ▾ 72.21.198.67/32

9. Identify or create the key pair

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- At instance launch, you specify an existing key pair *or* create a new key pair.
- A **key pair** consists of –
 - A **public key** that AWS stores.
 - A **private key** file that you store.
- It enables secure connections to the instance.
- For **Windows AMIs** –
 - Use the private key to obtain the administrator password that you need to log in to your instance.
- For **Linux AMIs** –
 - Use the private key to use SSH to securely connect to your instance.



mykey.pem



Amazon EC2 console view of a running EC2 instance

The screenshot displays the Amazon EC2 console interface. The left sidebar contains navigation links for EC2 Dashboard, Events, Tags, Reports, Limits, INSTANCES, Launch Templates, Spot Requests, Reserved Instances, Dedicated Hosts, Scheduled Instances, Capacity Reservations, IMAGES, AMIs, Bundle Tasks, ELASTIC BLOCK STORE, Volumes, and Snapshots. The main content area shows a list of instances with a search bar and a table of instance details. The instance i-092b6f3efba959a53 is selected, and its details are shown in the 'Description' tab.

Name	Instance ID	Instance Type	Instance State	Status Checks	Public DNS (IPv4)	IPv4 Public IP
	i-092b6f3efba959a53	t2.micro	running	Initializing	ec2-54-159-171-63.co...	54.159.171.63

Instance: **i-092b6f3efba959a53** Public DNS: **ec2-54-159-171-63.compute-1.amazonaws.com**

Description		Status Checks	Monitoring	Tags
Instance ID	i-092b6f3efba959a53			
Instance state	running			
Instance type	t2.micro			
Elastic IPs				
Availability zone	us-east-1c			
Security groups	launch-wizard-1. view inbound rules . view outbound rules			
Scheduled events	No scheduled events			
AMI ID	amzn2-ami-hvm-2.0.20190823.1-x86_64-gp2 (ami-0b69ea66ff7391e80)			
Platform	-			
Public DNS (IPv4)	ec2-54-159-171-63.compute-1.amazonaws.com			
IPv4 Public IP	54.159.171.63			
IPv6 IPs	-			
Private DNS	ip-172-31-82-44.ec2.internal			
Private IPs	172.31.82.44			
Secondary private IPs				
VPC ID	vpc-e4e9859e			
Subnet ID	subnet-d22779fc			
Network interfaces	eth0			



Another option: Launch an EC2 instance with the AWS Command Line Interface

- EC2 instances can also be created programmatically.



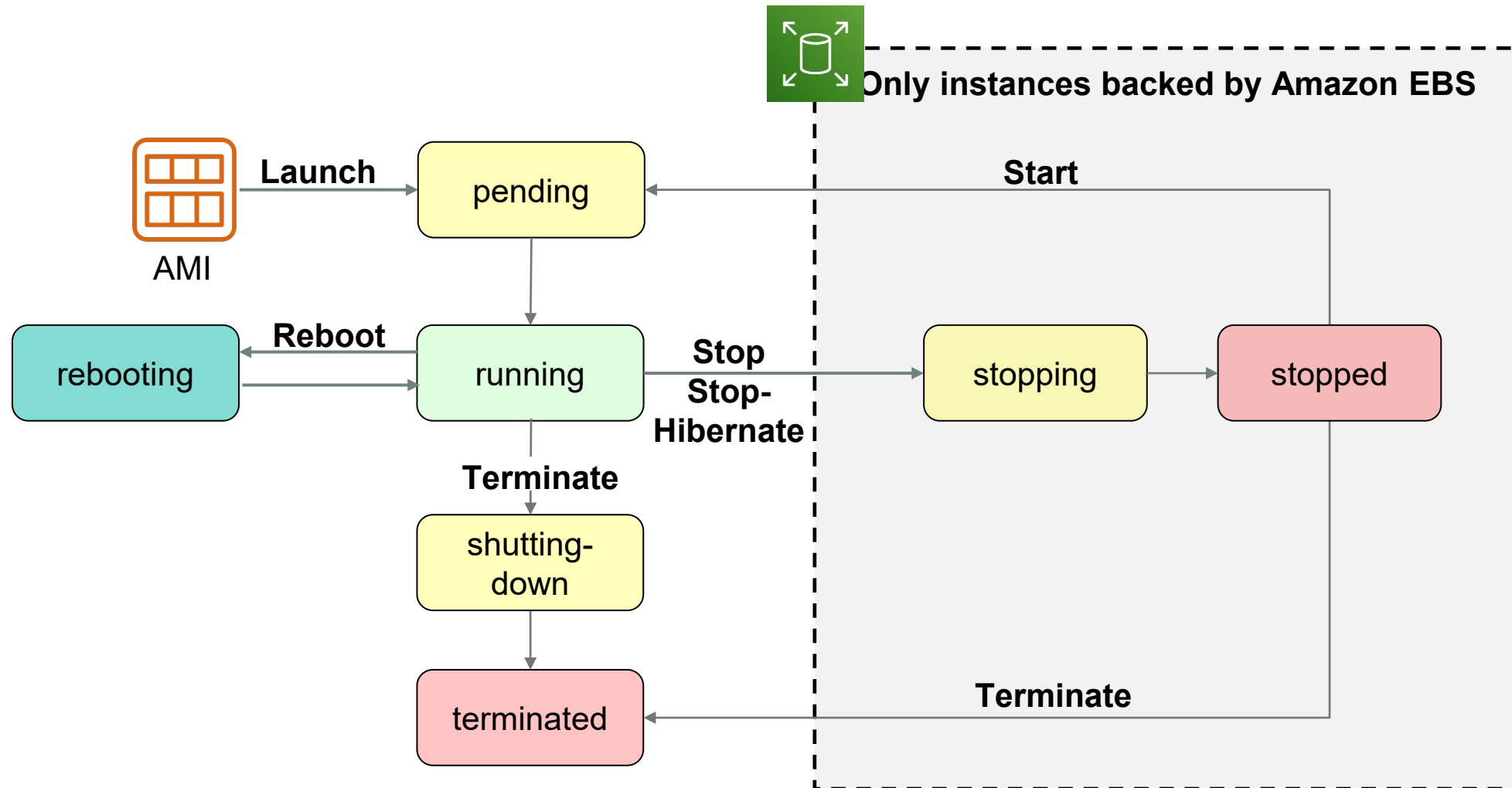
AWS Command Line
Interface (AWS CLI)

- This example shows how simple the command can be.
 - This command assumes that the key pair and security group already exist.
 - More options could be specified. See the [AWS CLI Command Reference](#) for details.

Example command:

```
aws ec2 run-instances \  
--image-id ami-1a2b3c4d \  
--count 1 \  
--instance-type c3.large \  
--key-name MyKeyPair \  
--security-groups MySecurityGroup \  
--region us-east-1
```

Amazon EC2 instance lifecycle





Consider using an Elastic IP address

- **Rebooting** an instance will *not* change any IP addresses or DNS hostnames.
- When an instance is **stopped** and then **started** again –
 - The *public* IPv4 address and *external* DNS hostname will change.
 - The *private* IPv4 address and internal DNS hostname do *not* change.
- If you require a persistent public IP address –
 - Associate an **Elastic IP address** with the instance.
- Elastic IP address characteristics –
 - Can be associated with instances in the Region as needed.
 - Remains allocated to your account until you choose to release it.



Elastic IP
Address



EC2 instance metadata

- **Instance metadata** is data about your instance.
- While you are connected to the instance, you can view it –
 - In a browser: `http://169.254.169.254/latest/meta-data/`
 - In a terminal window: `curl http://169.254.169.254/latest/meta-data/`
- Example retrievable values –
 - Public IP address, private IP address, public hostname, instance ID, security groups, Region, Availability Zone.
 - Any user data specified at instance launch can also be accessed at: `http://169.254.169.254/latest/user-data/`
- It can be used to configure or manage a running instance.
 - For example, author a configuration script that reads the metadata and uses it to configure applications or OS settings.

Amazon CloudWatch for monitoring

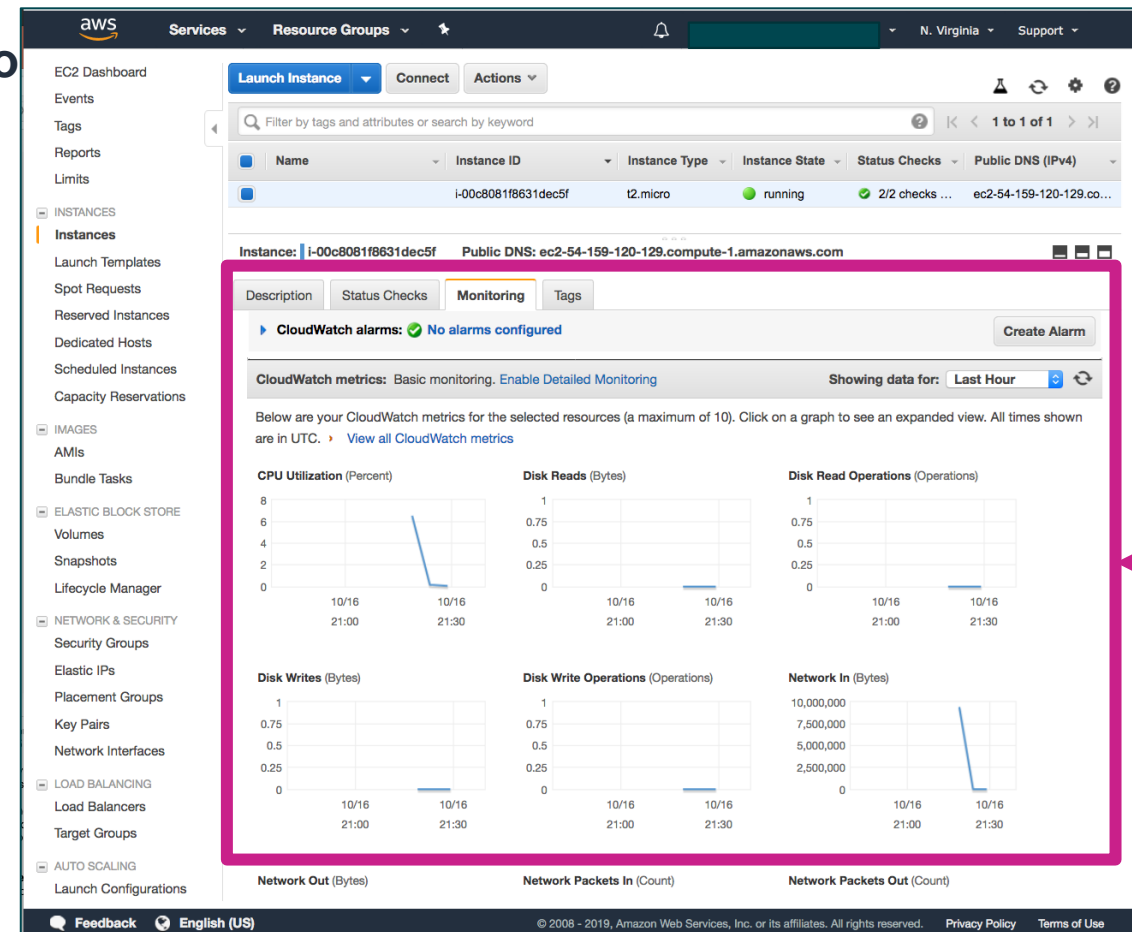
- Use **Amazon CloudWatch** to monitor EC2 instances
 - Provides near-real-time metrics
 - Provides charts in the Amazon EC2 console **Monitoring** tab that you can view
 - Maintains 15 months of historical data
- **Basic monitoring**
 - Default, no additional cost
 - Metric data sent to CloudWatch every 5 minutes
- **Detailed monitoring**
 - Fixed monthly rate for seven pre-selected metrics
 - Metric data delivered every 1 minute



Amazon CloudWatch



Instance with CloudWatch



Section 2 key takeaways



- **Amazon EC2** enables you to run Windows and Linux **virtual machines** in the cloud.
- You launch **EC2 instances** from an **AMI** template into a VPC in your account.
- You can choose from many **instance types**. Each instance type offers different combinations of CPU, RAM, storage, and networking capabilities.
- You can configure **security groups** to control access to instances (specify allowed ports and source).
- **User data** enables you to specify a script to run the first time that an instance launches.
- Only **instances that are backed by Amazon EBS** can be **stopped**.
- You can use **Amazon CloudWatch** to capture and review metrics on EC2 instances.

Recorded Amazon EC2 demonstration



Set up demo

Amazon Elastic Compute Cloud
(Amazon EC2)

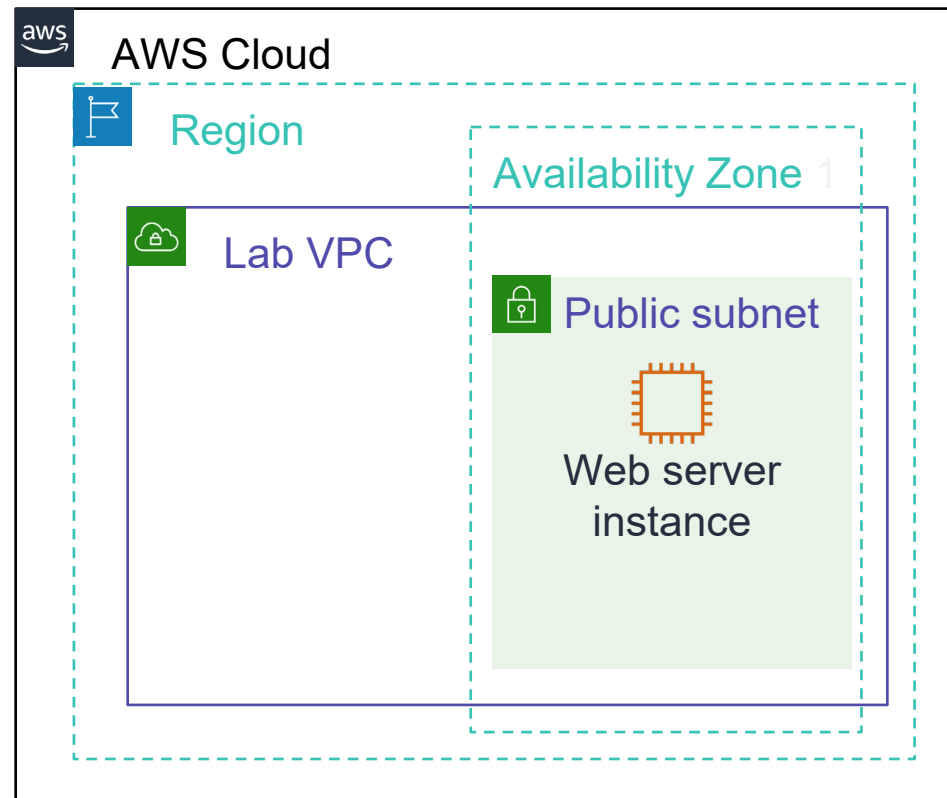
Lab 3:

Introduction to Amazon EC2



Lab 3 scenario

In this lab, you will launch and configure your first virtual machine that runs on Amazon EC2.





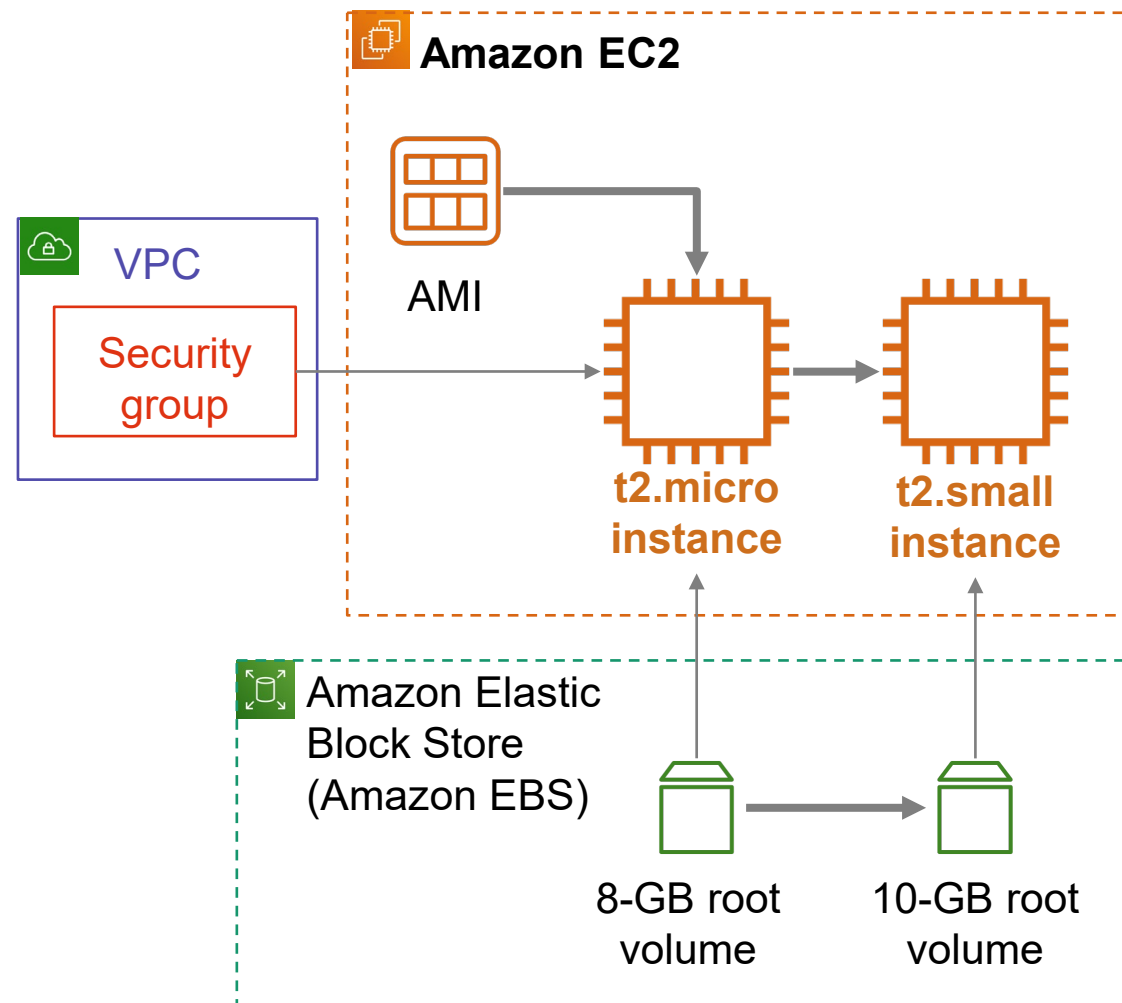
Lab 3: Tasks

- Task 1 – Launch Your Amazon EC2 Instance
- Task 2 – Monitor Your Instance
- Task 3 – Update Your Security Group and Access the Web Server
- Task 4 – Resize Your Instance: Instance Type and EBS Volume
- Task 5 – Explore EC2 Limits
- Task 6 – Test Termination Protection

Lab 3: Final product

By the end of the lab, you will have:

1. Launched an instance that is configured as a web server
2. Viewed the instance system log
3. Reconfigured a security group
4. Modified the instance type and root volume size





~ 35 minutes



Begin Lab 3: Introduction to Amazon EC2

Lab debrief: Key takeaways

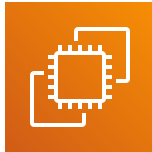


Activity: Amazon EC2

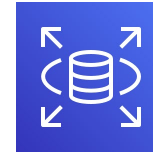


Photo by Pixabay from Pexels.

Activity: Gather information



Amazon EC2

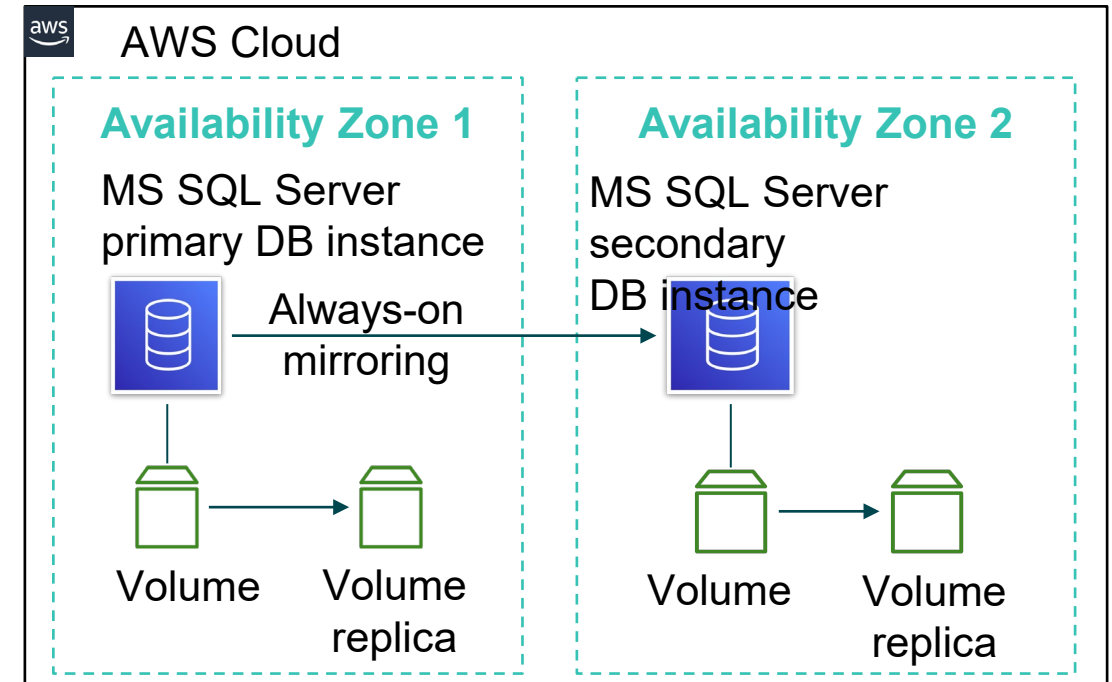
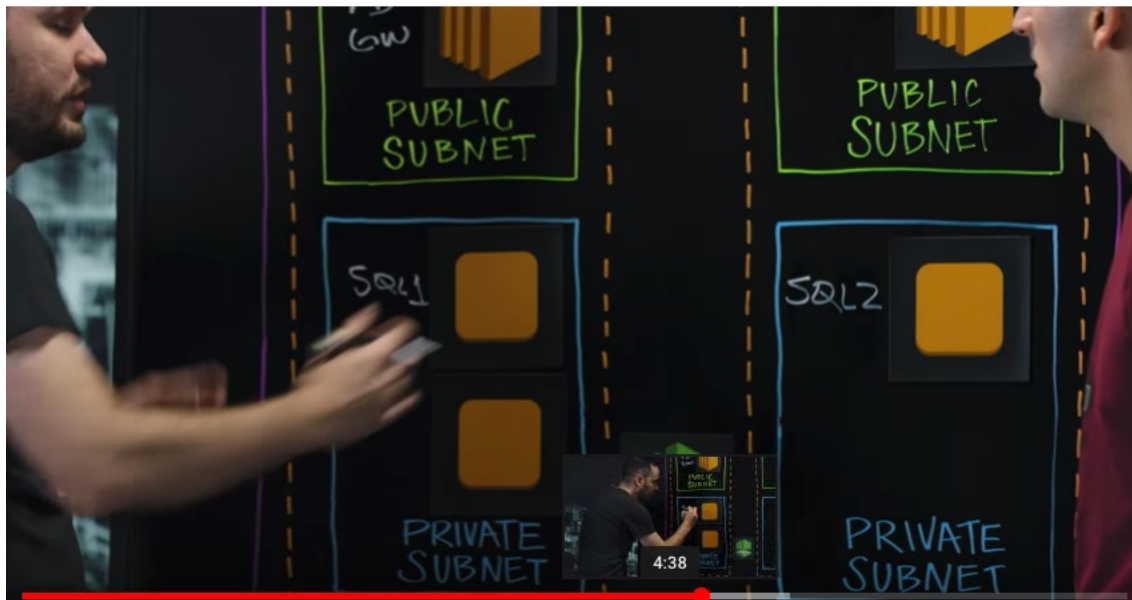


Amazon RDS



AWS Quick Starts

Automated, gold-standard deployments in the AWS Cloud





Activity: Check your understanding

1. Between Amazon EC2 or Amazon RDS, which provides a managed service? What does *managed service* mean?
2. Name at least one advantage of deploying Microsoft SQL Server on Amazon EC2 instead of Amazon RDS.
3. What advantage does the Quick Start provide over a manual installation on Amazon EC2?
4. Which deployment option offers the best approach for all use cases?
5. Which approach costs more: using Amazon EC2 or using Amazon RDS?



Section 3: Amazon EC2 cost optimization

Module 6: Compute





Amazon EC2 pricing models

On-Demand Instances

- Pay by the hour
- No long-term commitments.
- Eligible for the [AWS Free Tier](#).

Dedicated Hosts

- A physical server with EC2 instance capacity fully dedicated to your use.

Dedicated Instances

- Instances that run in a VPC on hardware that is dedicated to a single customer.

Reserved Instances

- Full, partial, or no upfront payment for instance you reserve.
- Discount on hourly charge for that instance.
- 1-year or 3-year term.

Scheduled Reserved Instances

- Purchase a capacity reservation that is always available on a recurring schedule you specify.
- 1-year term.

Spot Instances

- Instances run as long as they are available and your bid is above the Spot Instance price.
- They can be interrupted by AWS with a 2-minute notification.
- Interruption options include terminated, stopped or hibernated.
- Prices can be significantly less expensive compared to On-Demand Instances
- Good choice when you have flexibility in when your applications can run.

Per second billing available for On-Demand Instances, Reserved Instances, and Spot Instances that run Amazon Linux or Ubuntu.

Amazon EC2 pricing models: Benefits



On-Demand Instances	Spot Instances	Reserved Instances	Dedicated Hosts
Low cost and flexibility	Large scale, dynamic workload	Predictability ensures compute capacity is available when needed	- Save money on licensing costs - Help meet compliance and regulatory requirements

Amazon EC2 pricing models: Use cases



Spiky Workloads



Time-Insensitive Workloads



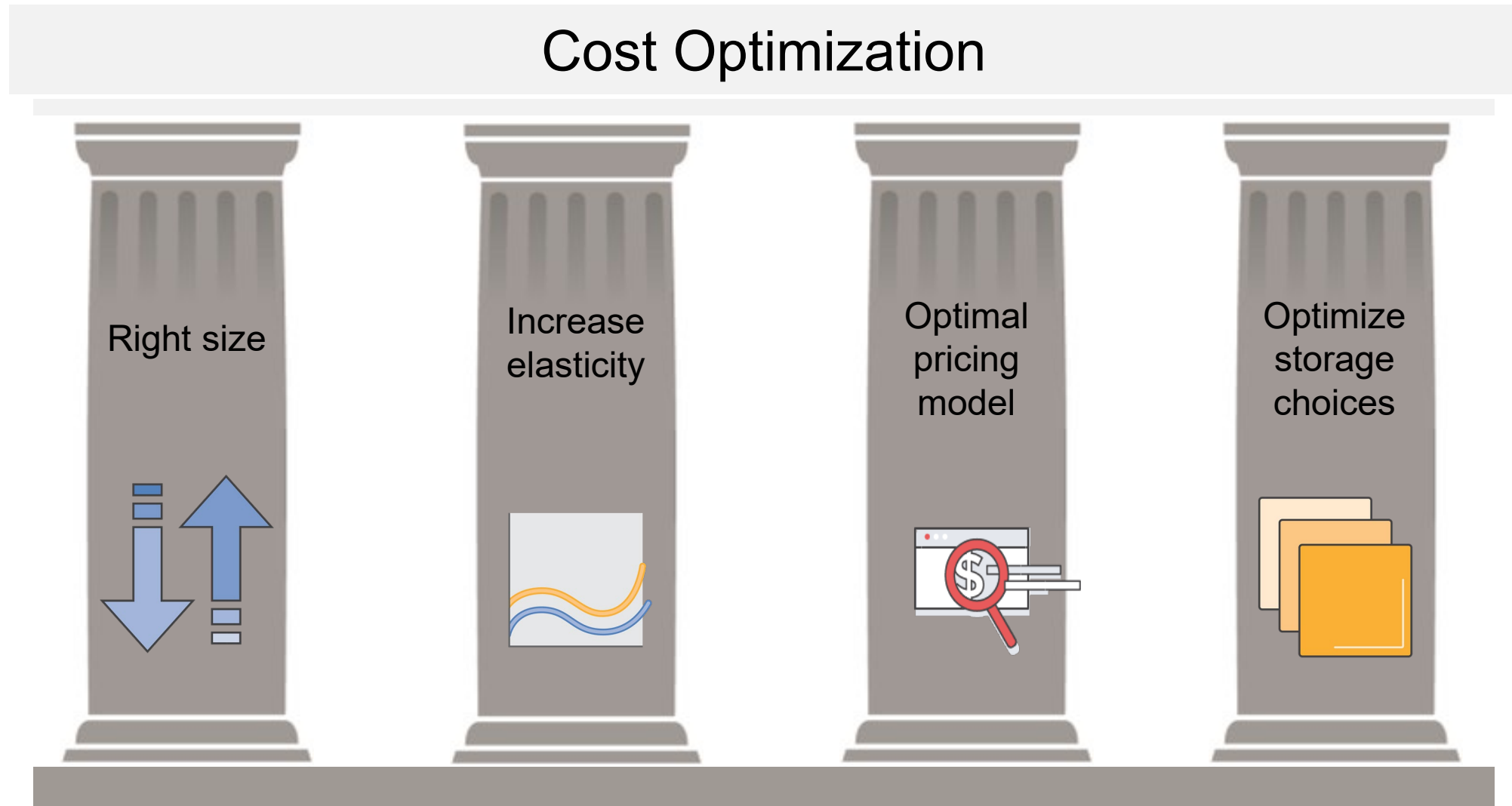
Steady-State Workloads



Highly Sensitive Workloads

On-Demand Instances	Spot Instances	Reserved Instances	Dedicated Hosts
• Short-term, spiky, or unpredictable workloads • Application development or testing	• Applications with flexible start and end times • Applications only feasible at very low compute prices • Users with urgent computing needs for large amounts of additional capacity	• Steady state or predictable usage workloads • Applications that require reserved capacity, including disaster recovery • Users able to make upfront payments to reduce total computing costs even further	• Bring your own license (BYOL) • Compliance and regulatory restrictions • Usage and licensing tracking • Control instance placement

The four pillars of cost optimization

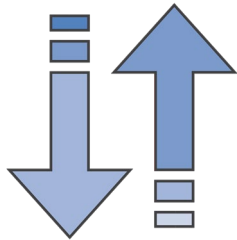




Pillar 1: Right size

Pillars

- 1. Right size
- 2. Increase elasticity
- 3. Optimal pricing model
- 4. Optimize storage choices



✓ Provision instances to match the need

- CPU, memory, storage, and network throughput
- Select appropriate **instance types** for your use

✓ Use Amazon CloudWatch metrics

- How idle are instances? When?
- Downsize instances

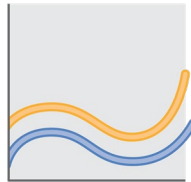
✓ Best practice: Right size, then reserve



Pillar 2: Increase elasticity

Pillars

- 1. Right-Size
- 2. Increase Elasticity
- 3. Optimal pricing model
- 4. Optimize storage choices



✓ **Stop** or **hibernate** Amazon EBS-backed instances that are not actively in use

- Example: non-production development or test instances

✓ Use **automatic scaling** to match needs based on usage

- Automated and time-based elasticity

Pillar 3: Optimal pricing model

Pillars

- 1. Right-Size
- 2. Increase Elasticity
- 3. Optimal pricing model
- 4. Optimize storage choices



✓Leverage the right pricing model for your use case

- Consider your usage patterns

✓Optimize and *combine* purchase types

✓Examples:

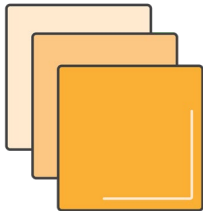
- Use **On-Demand Instance** and **Spot Instances** for variable workloads
- Use **Reserved Instances** for predictable workloads

✓Consider serverless solutions (AWS Lambda)

Pillar 4: Optimize storage choices

Pillars

- 1. Right-Size
- 2. Increase Elasticity
- 3. Optimal pricing model
- 4. Optimize storage choices

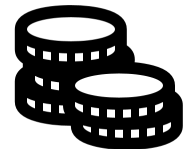
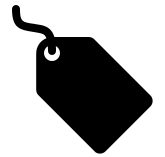


- ✓ Reduce costs while maintaining storage performance and availability
- ✓ Resize EBS volumes
- ✓ Change EBS volume types
 - ✓ Can you meet performance requirements with less expensive storage?
 - ✓ Example: **Amazon EBS Throughput Optimized HDD (st1)** storage typically costs half as much as the default **General Purpose SSD (gp2)** storage option.
- ✓ Delete EBS snapshots that are no longer needed
- ✓ Identify the most appropriate destination for specific types of data
 - ✓ Does the application need the instance to reside on Amazon EBS?
 - ✓ Amazon S3 storage options with lifecycle policies can reduce costs



Measure, monitor, and improve

- Cost optimization is an ongoing process.
- Recommendations –
 - Define and enforce **cost allocation tagging**.
 - Define metrics, set targets, and review regularly.
 - Encourage teams to **architect for cost**.
 - Assign the responsibility of optimization to an individual or to a team.



Section 3 key takeaways



- **Amazon EC2 pricing models** include On-Demand Instances, Reserved Instances, Spot Instances, Dedicated Instances, and Dedicated Hosts.
- **Spot Instances** can be interrupted with a 2-minute notification. However, they can offer significant cost savings over On-Demand Instances.
- The **four pillars of cost optimization** are:
 - Right size
 - Increase elasticity
 - Optimal pricing model
 - Optimize storage choices



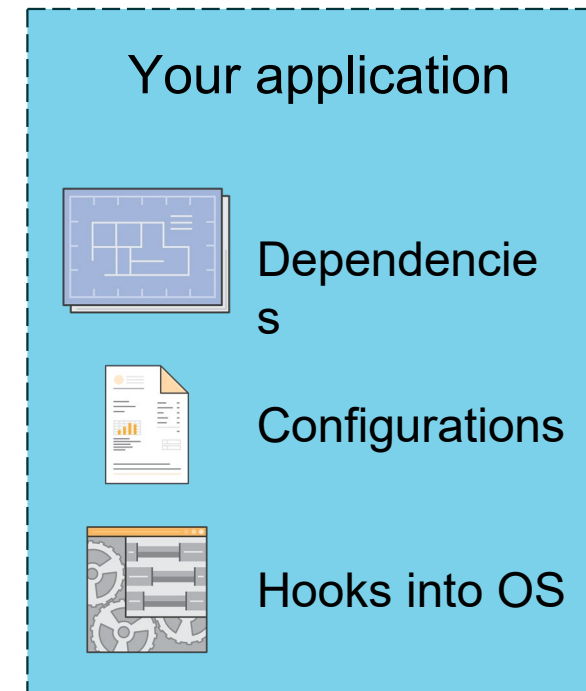
Section 4: Container services

Module 6: Compute

Container basics

- **Containers** are a method of operating system virtualization.
- **Benefits** –
 - Repeatable.
 - Self-contained environments.
 - Software runs the same in different environments.
 - Developer's laptop, test, production.
 - Faster to launch and stop or terminate than virtual machines

Your Container



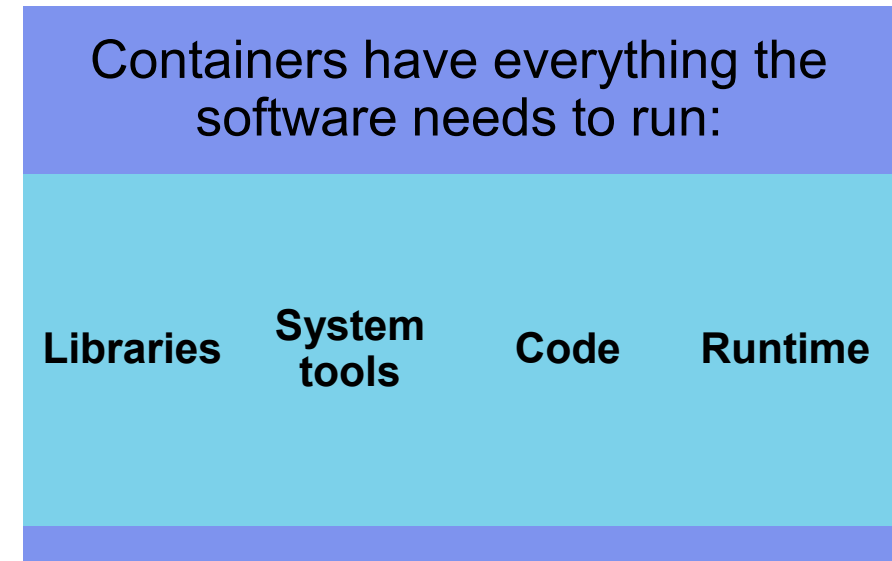


What is Docker?

- **Docker** is a software platform that enables you to build, test, and deploy applications quickly.
- You run containers on Docker.
 - Containers are created from a template called an *image*.
- A **container** has everything a software application needs to run.



Container

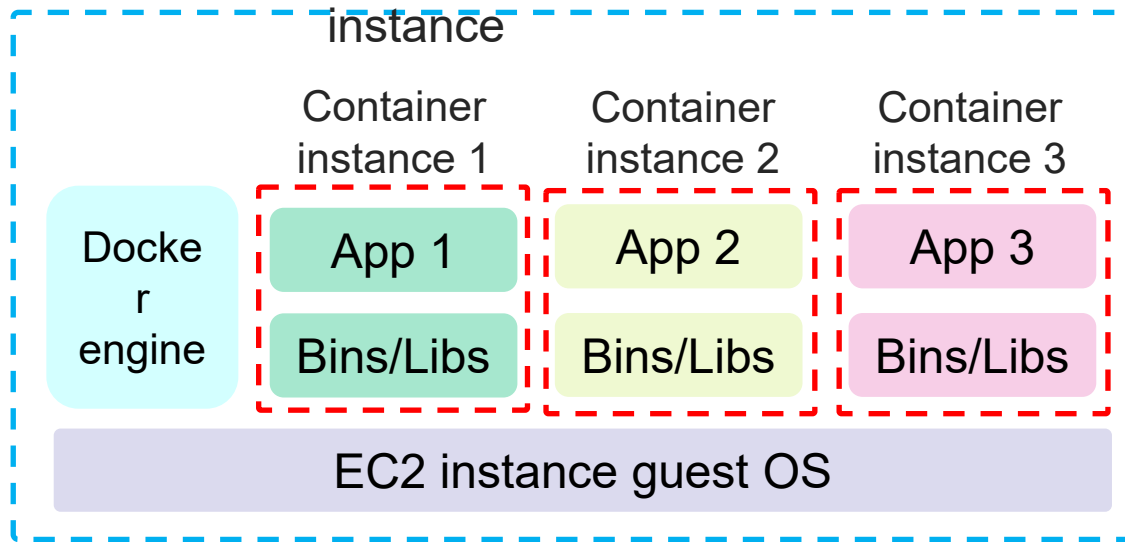


Containers versus virtual machines

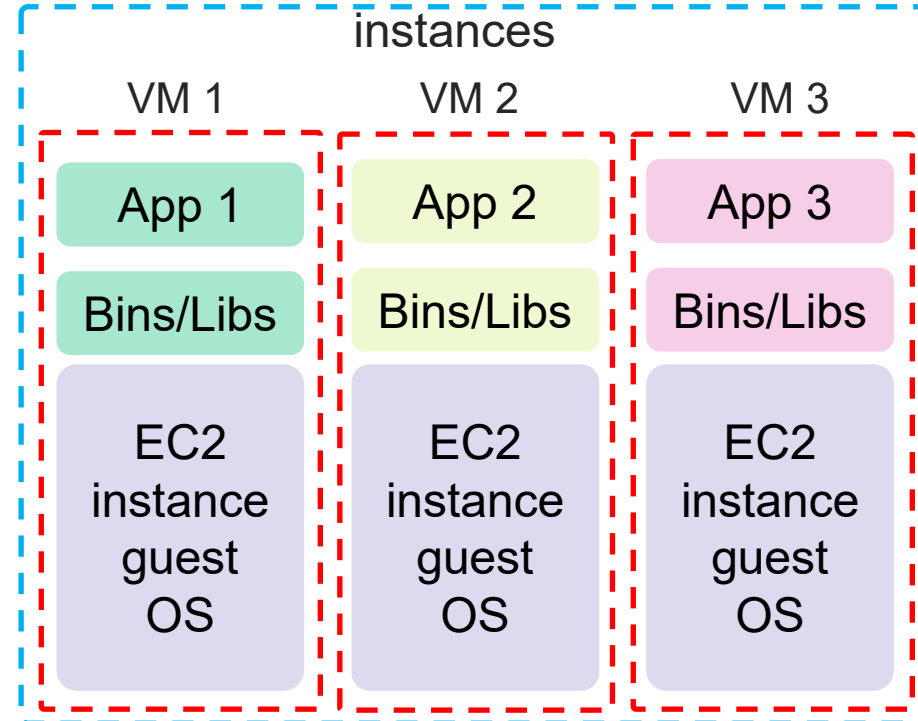
Example

e

Three containers on one EC2



Three virtual machines on three EC2 instances



Hypervisor

Host operating system

Physical server

Part of
AWS Global
Infrastructure



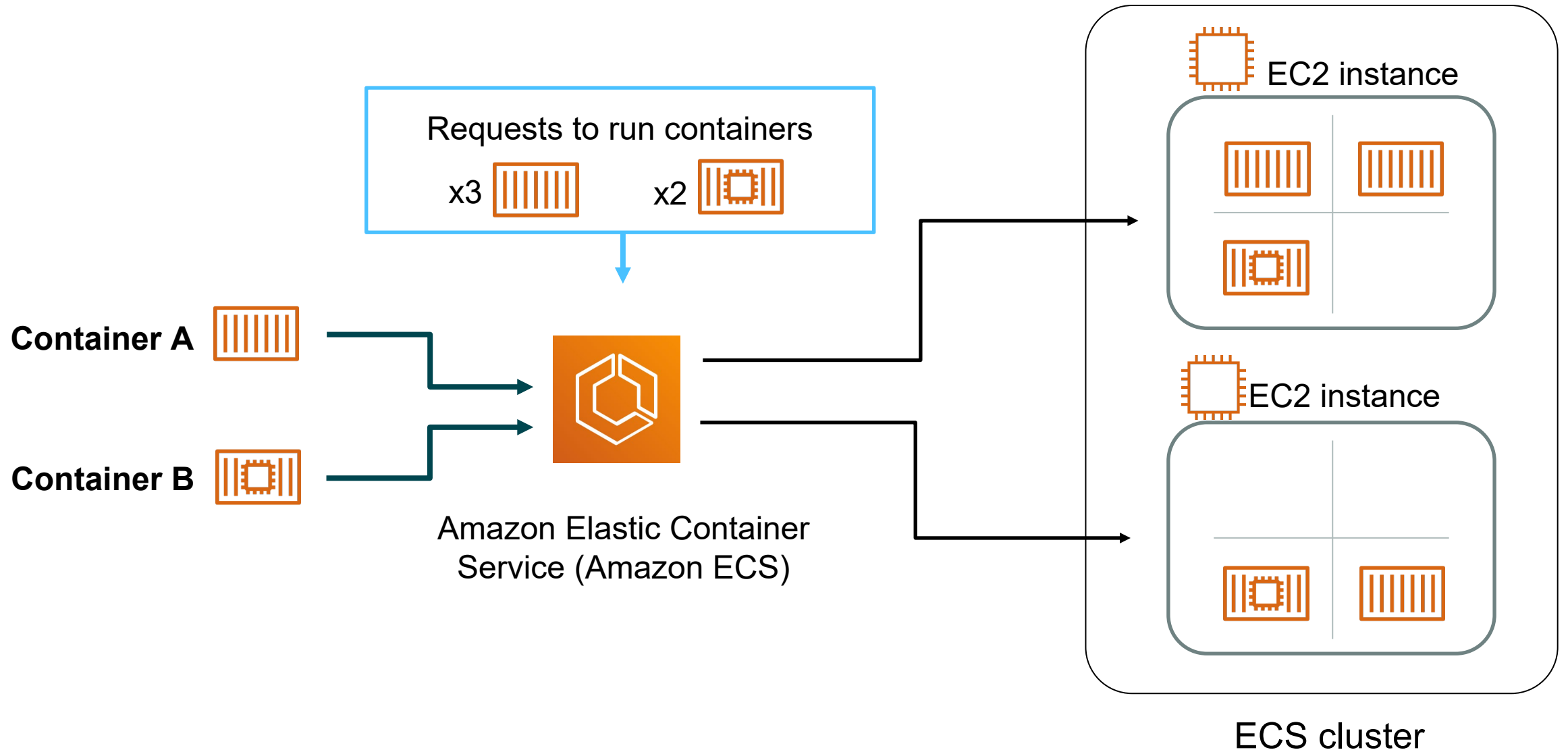
Amazon Elastic Container Service (Amazon ECS)

- Amazon Elastic Container Service (**Amazon ECS**) –
 - A highly scalable, fast, **container management service**
- Key benefits –
 - Orchestrates the running of Docker containers
 - Maintains and scales the fleet of nodes that run your containers
 - Removes the complexity of standing up the infrastructure
- Integrated with features that are familiar to Amazon EC2 service users –
 - Elastic Load Balancing
 - Amazon EC2 security groups
 - Amazon EBS volumes
 - IAM roles



**Amazon Elastic
Container Service**

Amazon ECS orchestrates containers

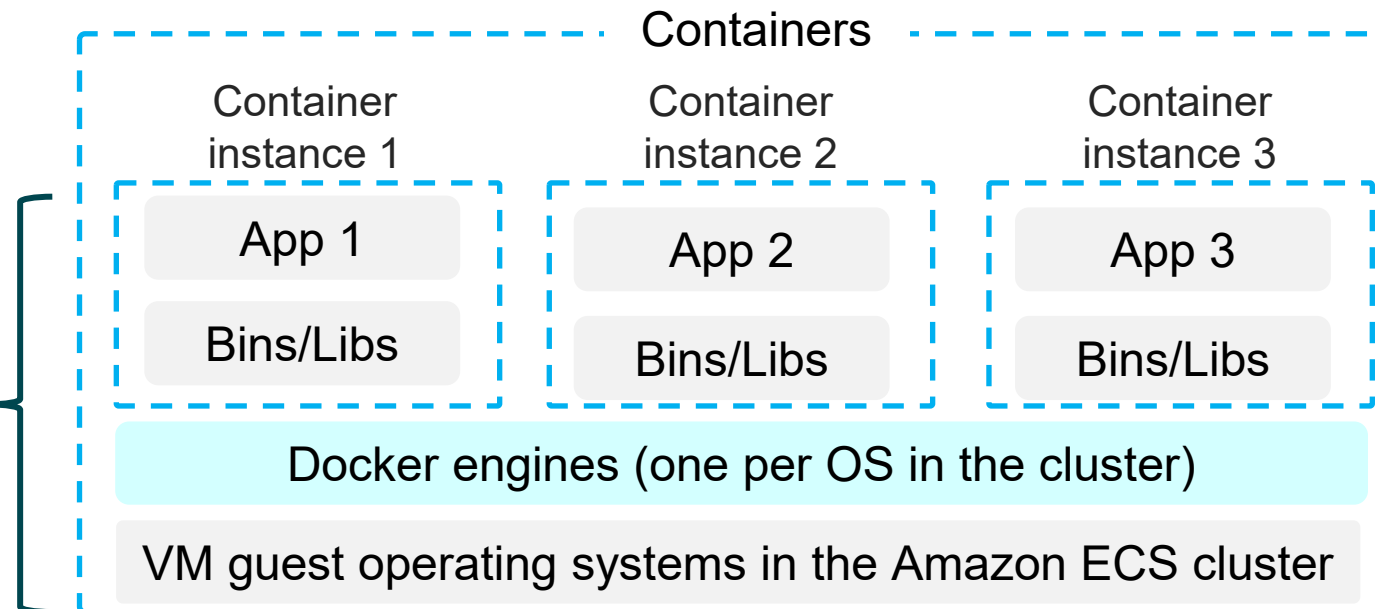


Amazon ECS cluster options

- **Key question:** Do ***you*** want to manage the Amazon ECS cluster that runs the containers?
- If **yes**, create an **Amazon ECS cluster backed by Amazon EC2** (provides more granular control over infrastructure)
- If **no**, create an **Amazon ECS cluster backed by AWS Fargate** (easier to maintain, focus on your applications)

**Amazon ECS
cluster backed
by Amazon EC2**

You manage



**Amazon ECS
cluster backed by
Fargate**

You manage

AWS manages



What is Kubernetes?

- Kubernetes is open source software for container orchestration.
 - Deploy and **manage containerized applications** *at scale*.
 - The same toolset can be used on premises and in the cloud.
- Complements Docker.
 - Docker enables you to run multiple containers on a single OS host.
 - Kubernetes **orchestrates** multiple Docker hosts (nodes).
- Automates –
 - Container provisioning.
 - Networking.
 - Load distribution.
 - Scaling.



Amazon Elastic Kubernetes Service (Amazon EKS)

- Amazon Elastic Kubernetes Service (**Amazon EKS**)
 - Enables you to run Kubernetes on AWS
 - Certified Kubernetes conformant (supports easy migration)
 - Supports Linux and Windows containers
 - Compatible with Kubernetes community tools and supports popular Kubernetes add-ons
- Use Amazon EKS to –
 - Manage clusters of Amazon EC2 compute instances
 - Run containers that are orchestrated by Kubernetes on those instances



**Amazon Elastic
Kubernetes Service**

Amazon Elastic Container Registry (Amazon ECR)

Amazon ECR is a fully managed Docker **container registry** that makes it easy for developers to store, manage, and deploy Docker container images.



**Amazon Elastic
Container Registry**



Image



Registry

Amazon ECS integration

Docker support

Team collaboration

Access control

Third-party integrations

Section 4 key takeaways



- **Containers** can hold everything that an application needs to run.
- **Docker** is a software platform that packages software into containers.
 - A single application can span multiple containers.
- Amazon Elastic Container Service (**Amazon ECS**) orchestrates the running of Docker containers.
- **Kubernetes** is open source software for container orchestration.
- Amazon Elastic Kubernetes Service (**Amazon EKS**) enables you to run Kubernetes on AWS
- Amazon Elastic Container Registry (**Amazon ECR**) enables you to store, manage, and deploy your Docker containers.

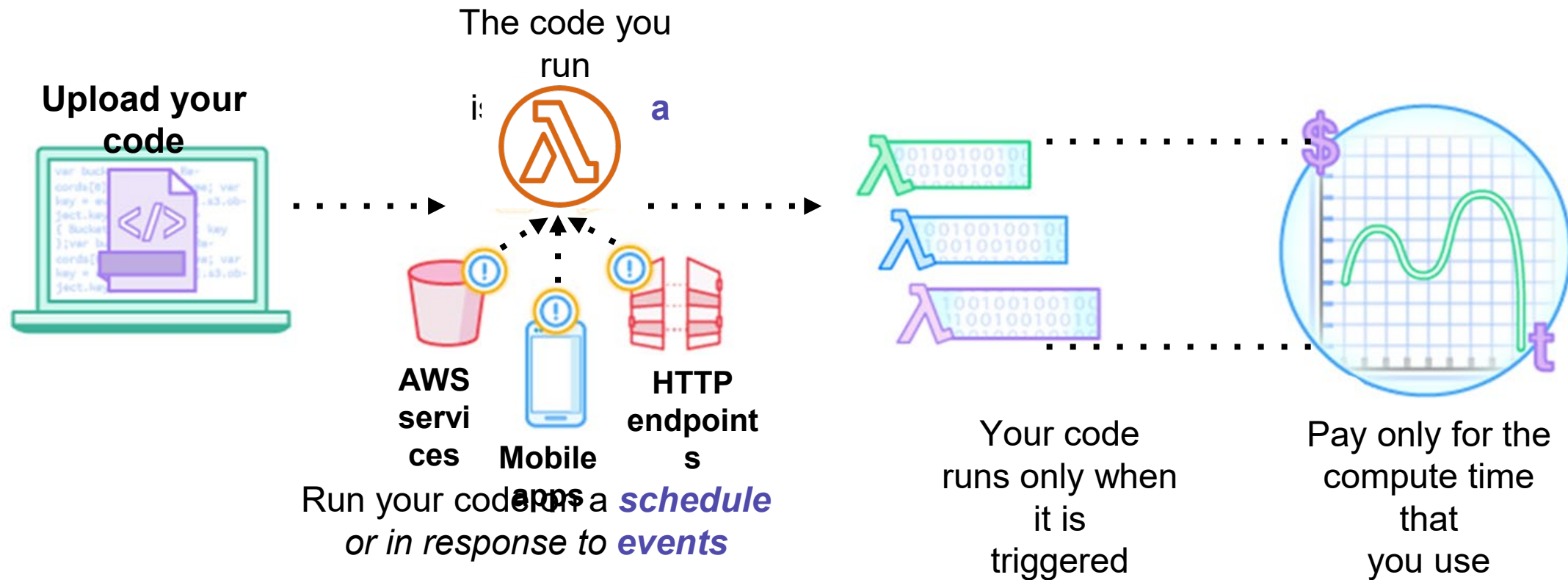


Section 5: Introduction to AWS Lambda

Module 6: Compute

AWS Lambda: Run code without servers

AWS Lambda is a **serverless** compute service.



Benefits of Lambda



**AWS
Lambda**



It supports multiple programming languages



Completely automated administration



Built-in fault tolerance



It supports the orchestration of multiple functions



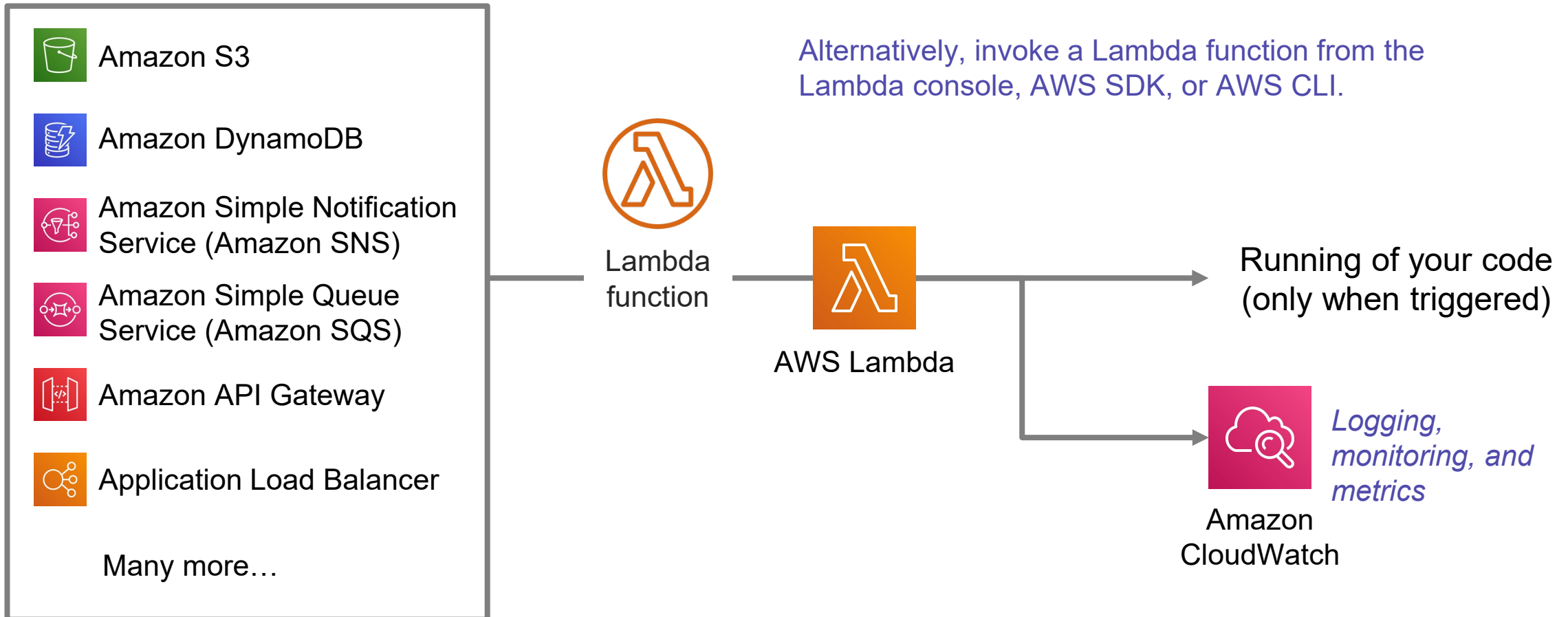
Pay-per-use pricing

AWS Lambda event sources

Event sources

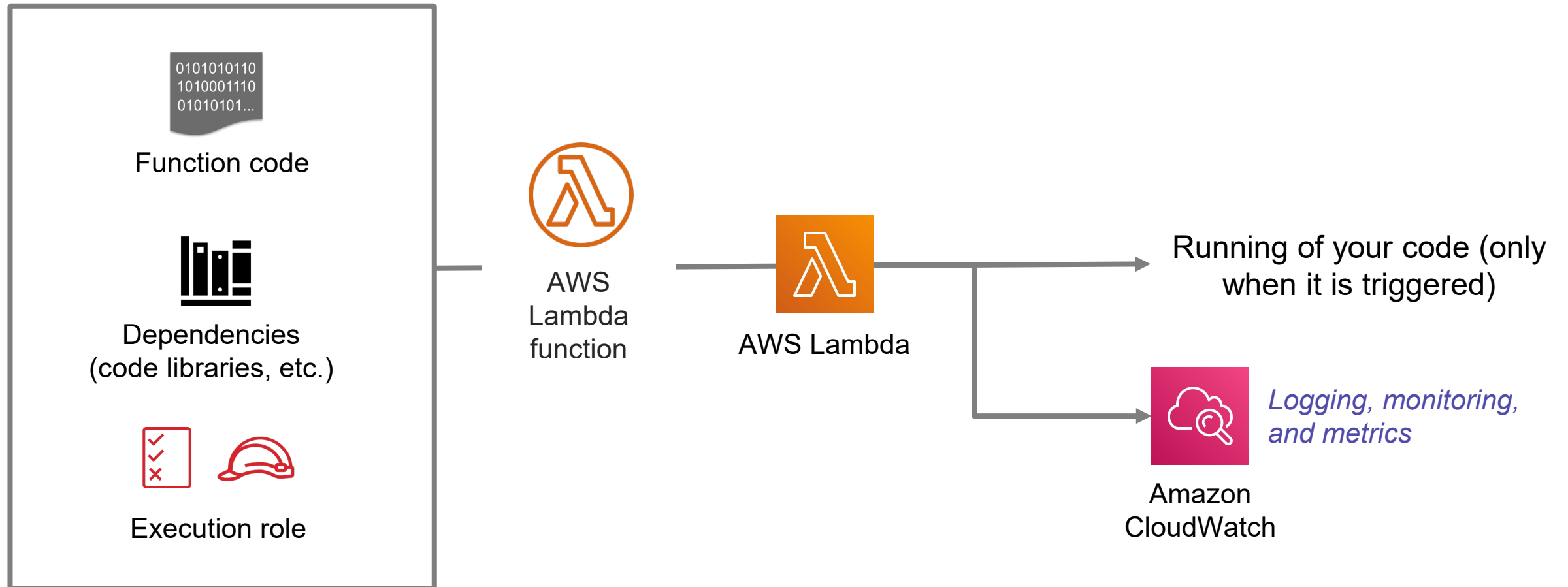
Configure other AWS services as **event sources** to invoke your function as shown here.

Alternatively, invoke a Lambda function from the Lambda console, AWS SDK, or AWS CLI.



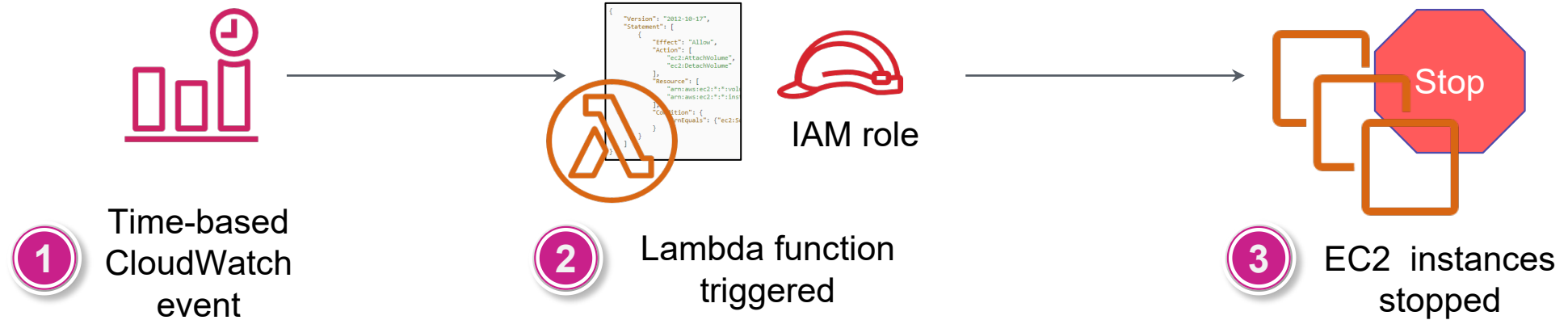
AWS Lambda function configuration

Lambda function configuration

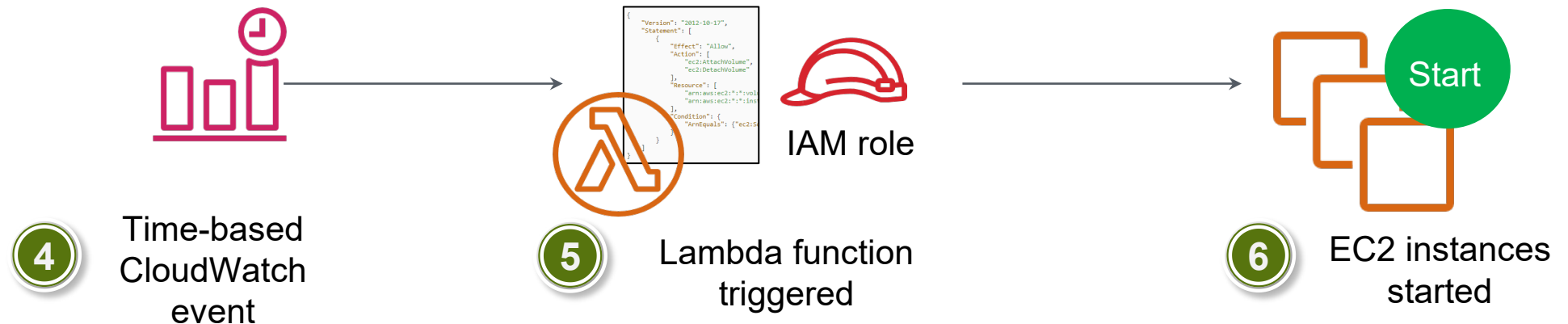


Schedule-based Lambda function example: Start and stop EC2 instances

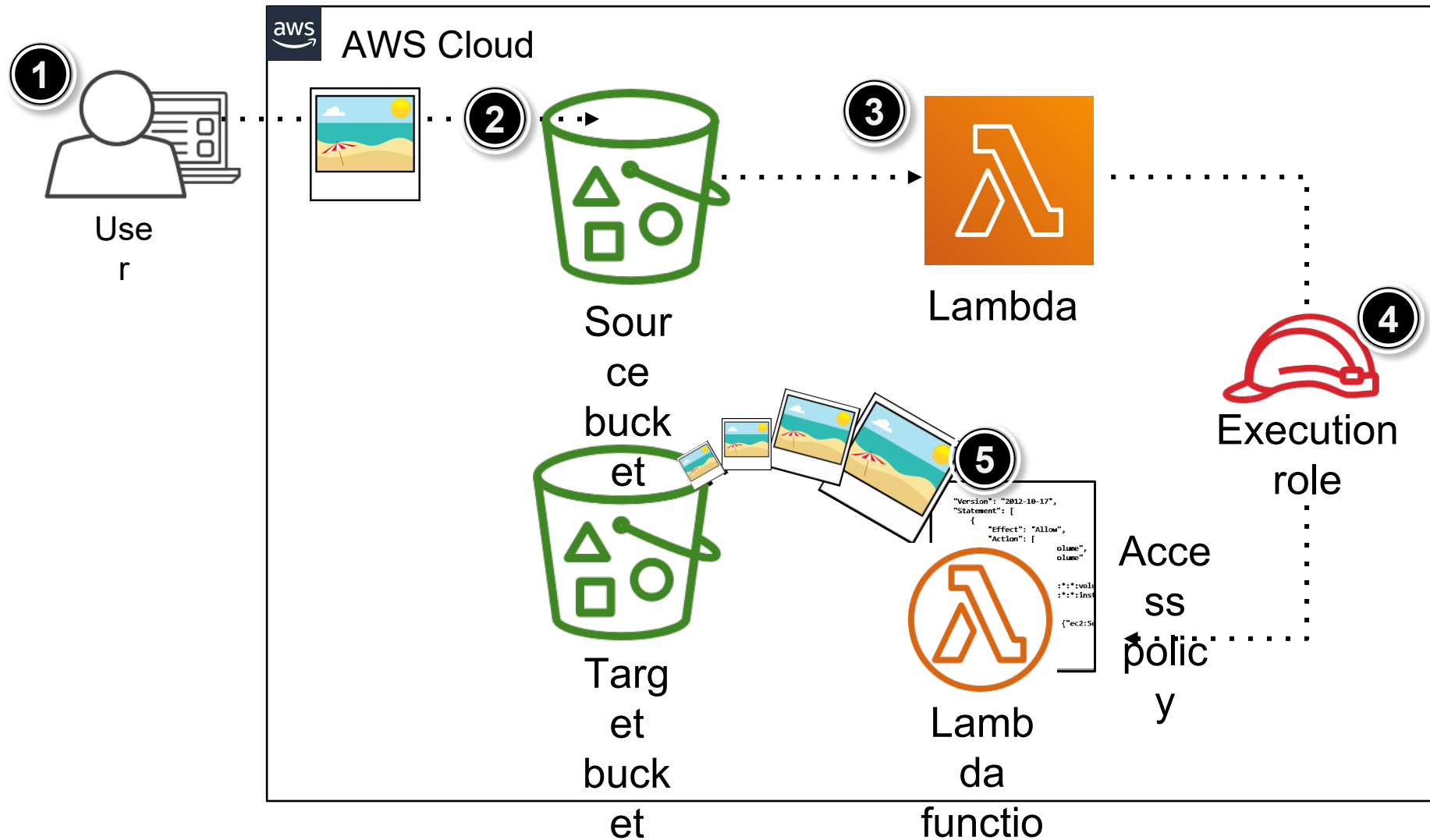
Stop instances example



Start instances example



Event-based Lambda function example: Create thumbnail images





AWS Lambda quotas

Soft limits per Region:

- Concurrent executions = 1,000
- Function and layer storage = 75 GB

Hard limits for individual functions:

- Maximum function memory allocation = 10,240 MB
- Function timeout = 15 minutes
- Deployment package size = 250 MB unzipped, including layers
- Container image code package size = 10 GB

Additional limits also exist. Details are in the AWS Lambda quotas documentation at <https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html>.

Section 5 key takeaways



- **Serverless computing** enables you to build and run applications and services without provisioning or managing servers.
- **AWS Lambda is a serverless compute service** that provides built-in fault tolerance and automatic scaling.
- An **event source** is an AWS service or developer-created application that triggers a Lambda function to run.
- The maximum memory allocation for a single Lambda function is 10,240 MB.
- The maximum run time for a Lambda function is 15 minutes.



Activity: Create an AWS Lambda Stopinator Function

To complete this activity:

- Go to the hands-on lab environment and launch the AWS Lambda activity.
- Follow the instructions that are provided in the hands-on lab environment.

Activity debrief: key takeaways





Section 6: Introduction to AWS Elastic Beanstalk

Module 6: Compute



AWS Elastic Beanstalk

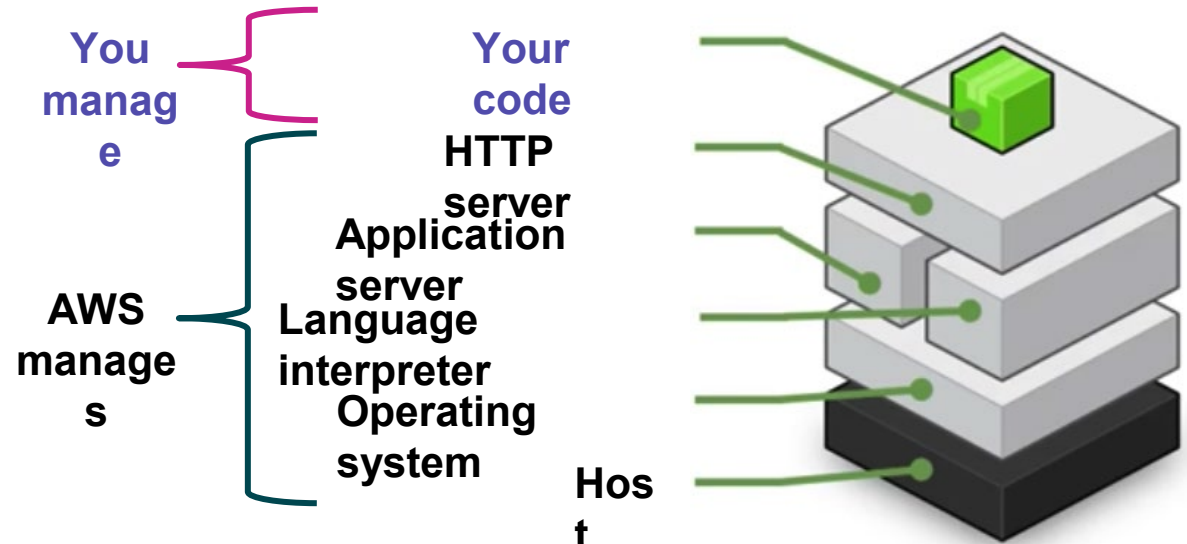


AWS Elastic Beanstalk

- An easy way to get **web applications** up and running
- A **managed service** that automatically handles –
 - Infrastructure provisioning and configuration
 - Deployment
 - Load balancing
 - Automatic scaling
 - Health monitoring
 - Analysis and debugging
 - Logging
- No additional charge for Elastic Beanstalk
 - Pay only for the underlying resources that are used

AWS Elastic Beanstalk deployments

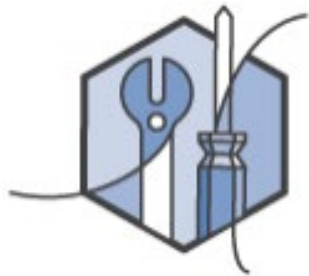
- It supports web applications written for common platforms
 - **Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker**
- You upload your code
 - Elastic Beanstalk automatically handles the deployment
 - Deploys on servers such as Apache, NGINX, Passenger, Puma, and Microsoft Internet Information Services (IIS)



Benefits of Elastic Beanstalk



Fast and simple to
start using



Developer
productivity



Difficult to
outgrow



Complete resource
control



Activity: AWS Elastic Beanstalk

To complete this activity:

- Go to the hands-on lab environment and launch the AWS Elastic Beanstalk activity.
- Follow the instructions that are provided in the hands-on lab environment.

Activity debrief: Key takeaways



Section 6 key takeaways



- **AWS Elastic Beanstalk** enhances developer productivity.
 - Simplifies the process of deploying your application.
 - Reduces management complexity.
- Elastic Beanstalk supports **Java**, **.NET**, **PHP**, **Node.js**, **Python**, **Ruby**, **Go**, and **Docker**
- There is no charge for Elastic Beanstalk. Pay only for the AWS resources that you use.



Module wrap-up

Module 6: Compute





Module summary

In summary, in this module, you learned how to:

- Provide an overview of different AWS compute services in the cloud
- Demonstrate why to use Amazon Elastic Compute Cloud (Amazon EC2)
- Identify the functionality in the Amazon EC2 console
- Perform basic functions in Amazon EC2 to build a virtual computing environment
- Identify Amazon EC2 cost optimization elements
- Demonstrate when to use AWS Elastic Beanstalk
- Demonstrate when to use AWS Lambda
- Identify how to run containerized applications in a cluster of managed servers

Complete the knowledge check





Sample exam question

Which AWS service helps developers quickly deploy resources which can make use of different programming languages, such as .NET and Java?

Choice	Response
A	AWS CloudFormation
B	AWS SQS
C	AWS Elastic Beanstalk
D	Amazon Elastic Compute Cloud (Amazon EC2)



Additional resources

- Amazon EC2 Documentation: <https://docs.aws.amazon.com/ec2/>
- Amazon EC2 Pricing: <https://aws.amazon.com/ec2/pricing/>
- Amazon ECS Workshop: <https://ecsworkshop.com/>
- Running Containers on AWS: <https://containeronaws.com/>
- Amazon EKS Workshop: <https://www.eksworkshop.com/>
- AWS Lambda Documentation: <https://docs.aws.amazon.com/lambda/>
- AWS Elastic Beanstalk Documentation: <https://docs.aws.amazon.com/elastic-beanstalk/>
- Cost Optimization Playbook:
https://d1.awsstatic.com/pricing/AWS_CO_Playbook_Final.pdf



Thank you



Corrections, feedback, or other questions?

Contact us at <https://support.aws.amazon.com/#/contacts/aws-academy>.

All trademarks are the property of their owners.